CampusShelf

Three-Tier Library Lending Application on AWS

Deployment Evidence

SE-490 Cloud Computing — Complex Engineering Activity

Student name: Muhammad Ahmed Saleem Roll. no: SE-23059

Student name: Shaheer Aqeel Roll. no: SE-23094

Contents

1. Network Configuration (VPC, Subnets, Routing)
2. Compute Instances / Container Hosts
3. Security Groups
4. Load Balancer Configuration
5. Database Configuration
6. Docker Build and Registry (Amazon ECR)
7. CI/CD Pipeline Execution (GitHub Actions)
8. Running Application in Browser
9. Infrastructure as Code (Terraform)
10. Secrets and Access Management
11. Monitoring, Alarms and Cost Control
12. Scalability and Reliability Testing

1. Network Configuration (VPC, Subnets, Routing)

The application runs in a custom VPC (10.0.0.0/16) in ap-southeast-2 with three subnet tiers across two Availability Zones: public (10.0.1.0/24, 10.0.2.0/24), private app (10.0.11.0/24, 10.0.12.0/24) and private database (10.0.21.0/24, 10.0.22.0/24).

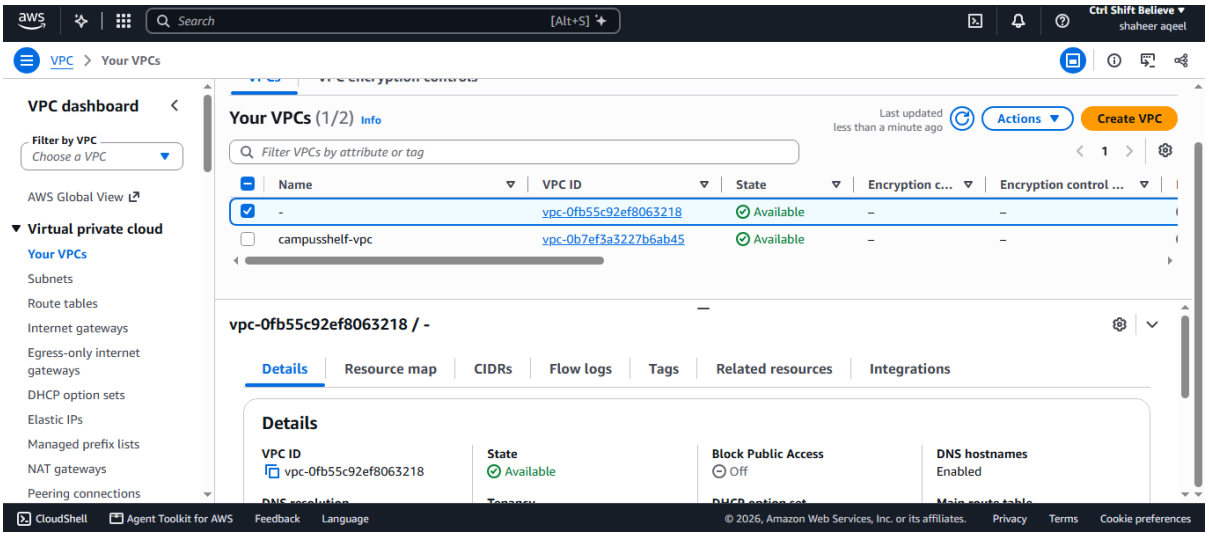


Figure 1: *VPC list showing the custom campusshelf-vpc (10.0.0.0/16) next to the default VPC*

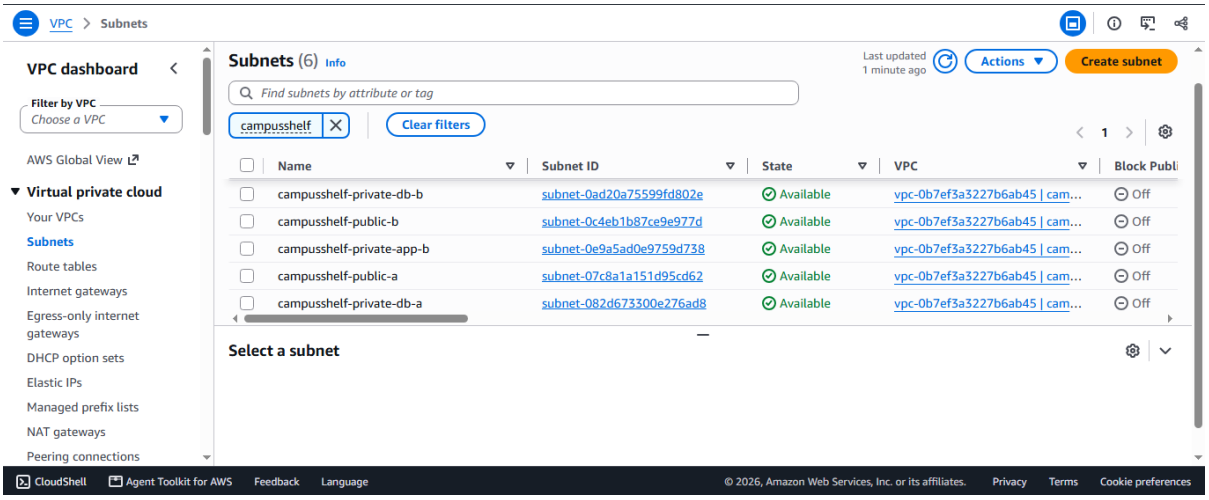


Figure 2: *The six CampusShelf subnets: public, private-app and private-db in two AZs*

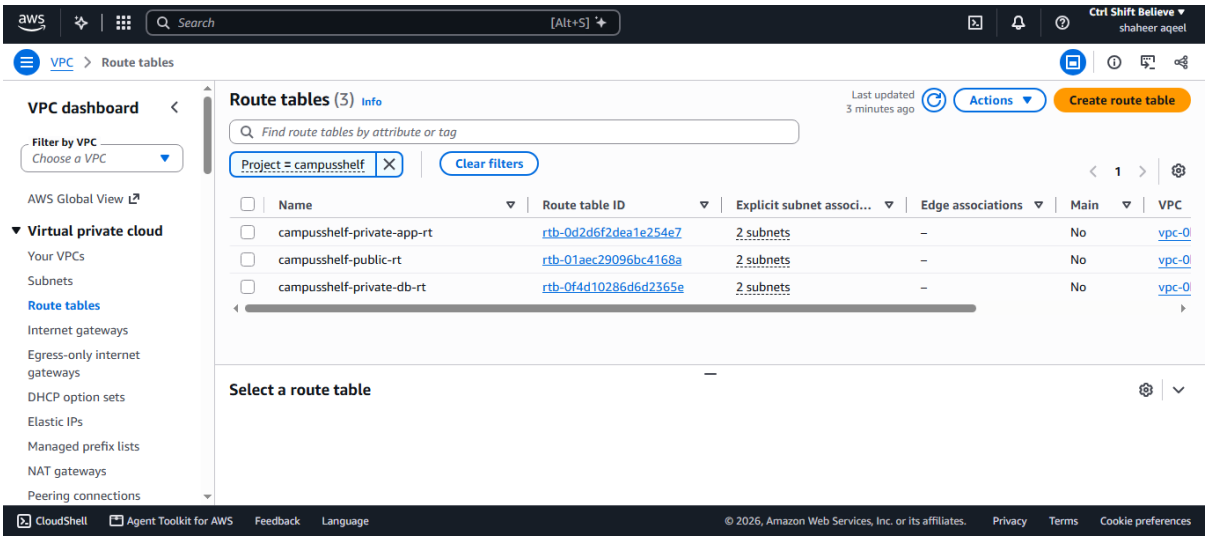


Figure 3: *Route tables for the public, private-app and private-db tiers*

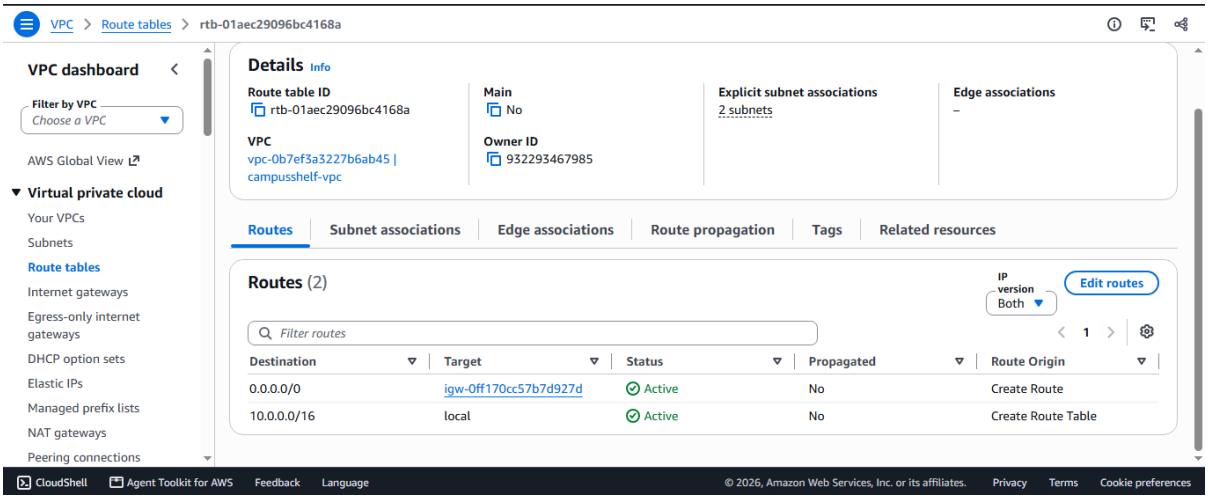


Figure 4: *Public route table: 0.0.0.0/0 goes to the Internet Gateway*

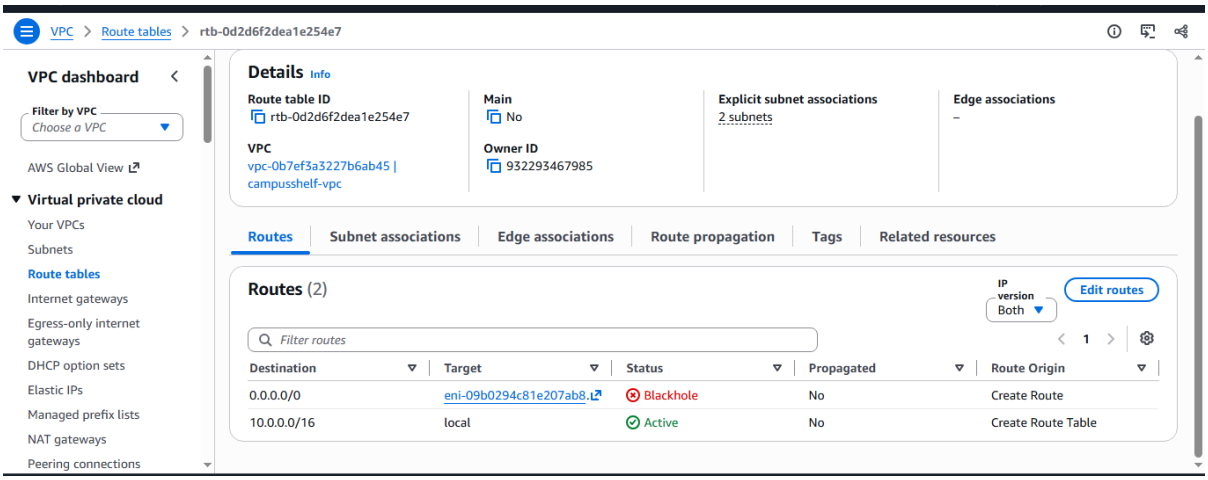


Figure 5: *Private-app route table: 0.0.0.0/0 goes to the NAT instance ENI (shown as Blackhole because the NAT instance was stopped to save cost)*

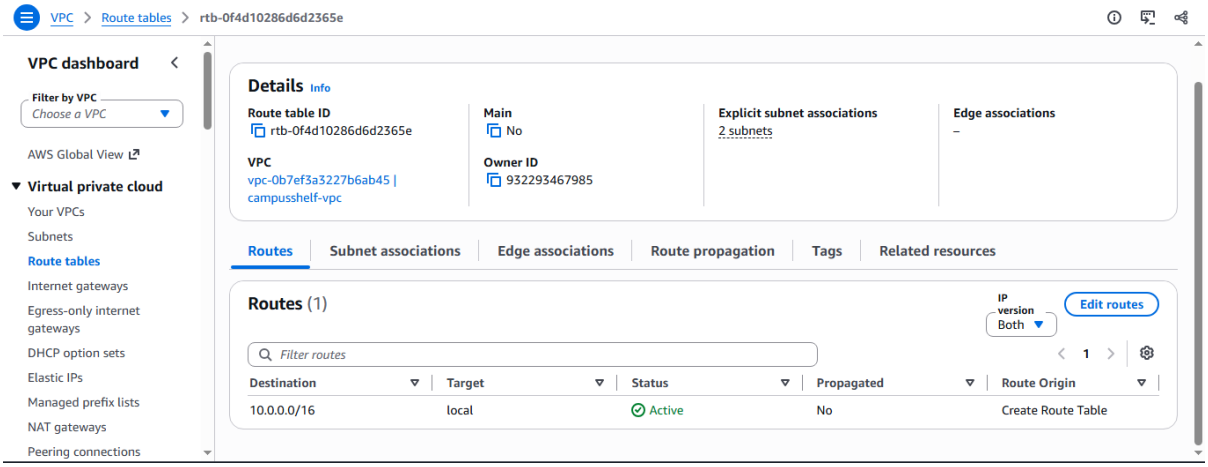


Figure 6: *Private-db route table: local VPC route only, so no internet access*

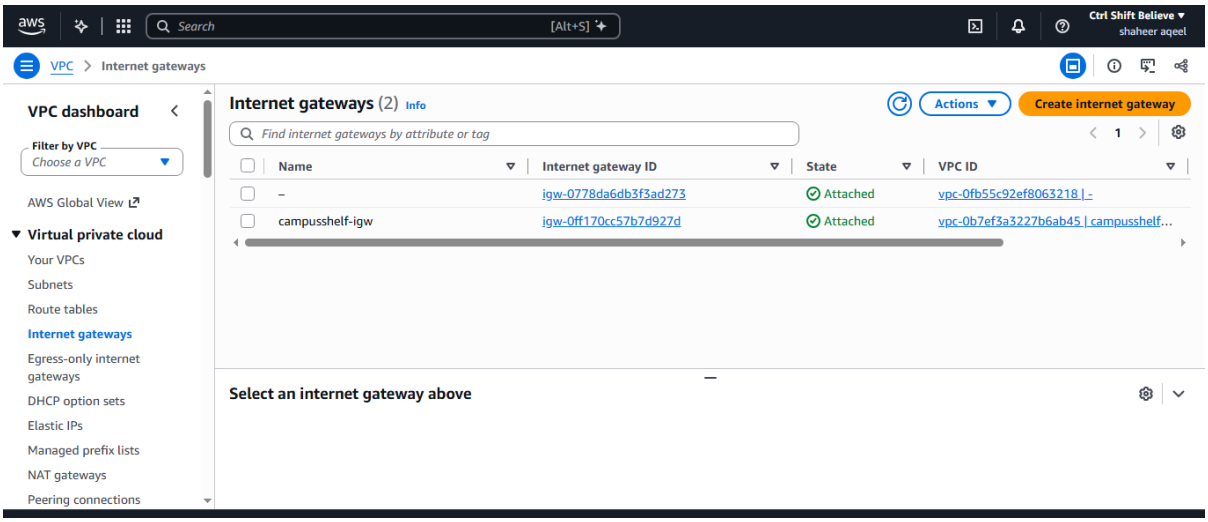


Figure 7: *Internet Gateway attached to campusshelf-vpc*

2. Compute Instances / Container Hosts

The web and app tiers run as Docker containers on t3.micro EC2 instances in Auto Scaling Groups in the private app subnets. A t3.micro NAT instance in the public subnet gives them outbound internet access (a free-tier alternative to a NAT Gateway).



Figure 8: *Auto Scaling Groups campusshelf-web and campusshelf-app: instances InService and Healthy*

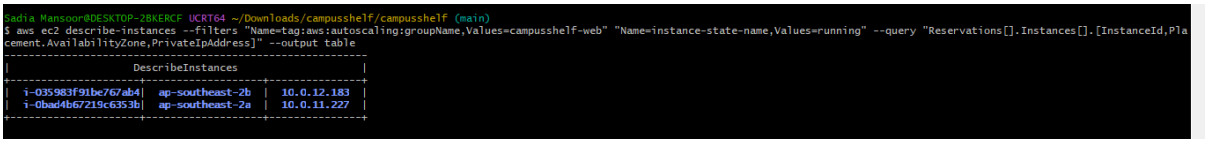


Figure 9: *Running web and app instances in the private subnets (10.0.11.x / 10.0.12.x)*

3. Security Groups

The security groups are chained so each tier only accepts traffic from the tier in front of it: internet → ALB (HTTP 80) → web (Nginx) → app (Express) → RDS (PostgreSQL 5432). The NAT group only accepts traffic from the private app subnets.

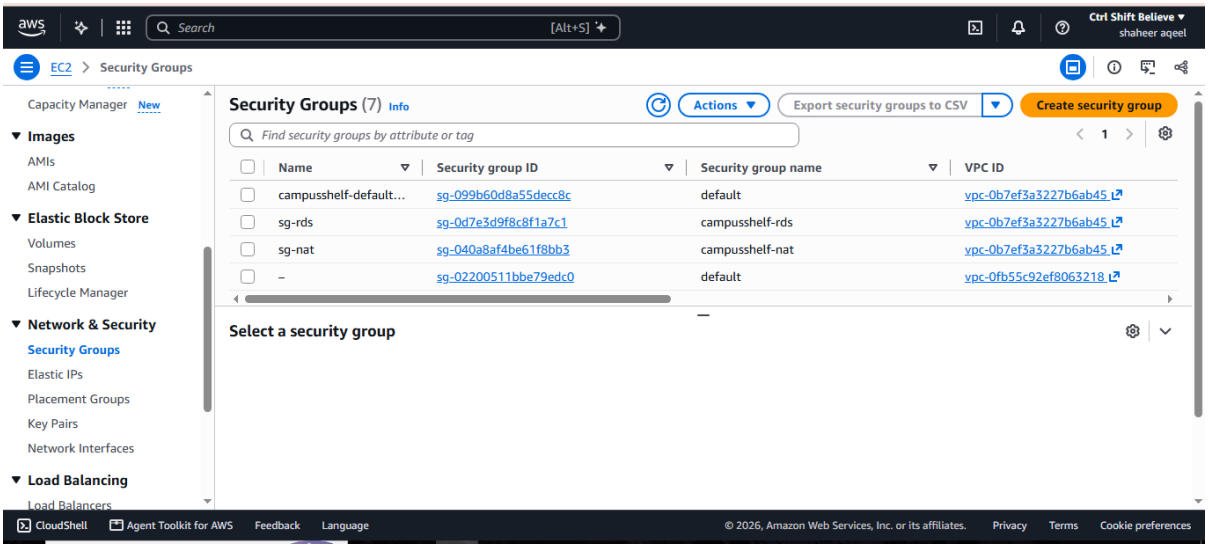


Figure 10: *Security groups of the CampusShelf VPC*

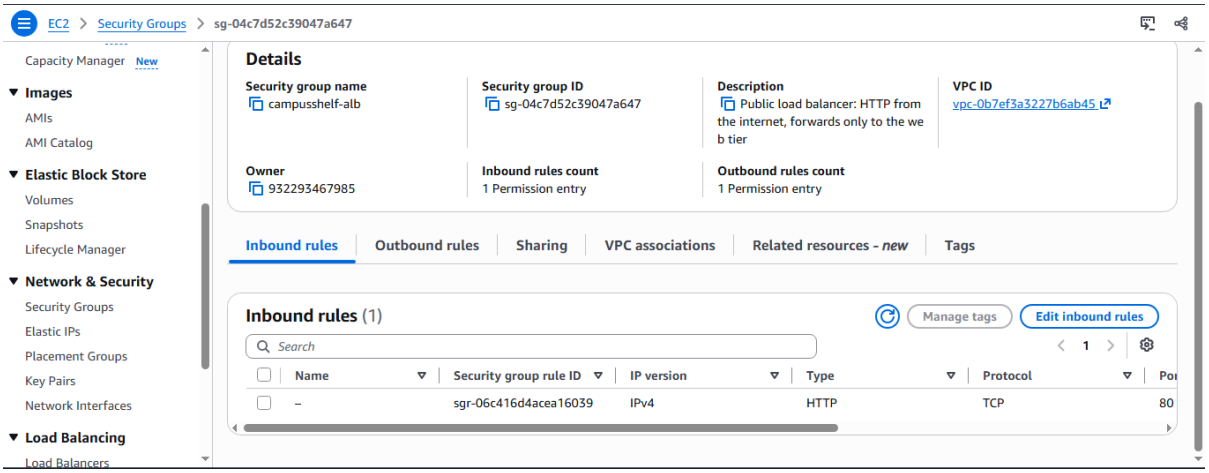


Figure 11: *ALB security group: inbound HTTP 80 from the internet*

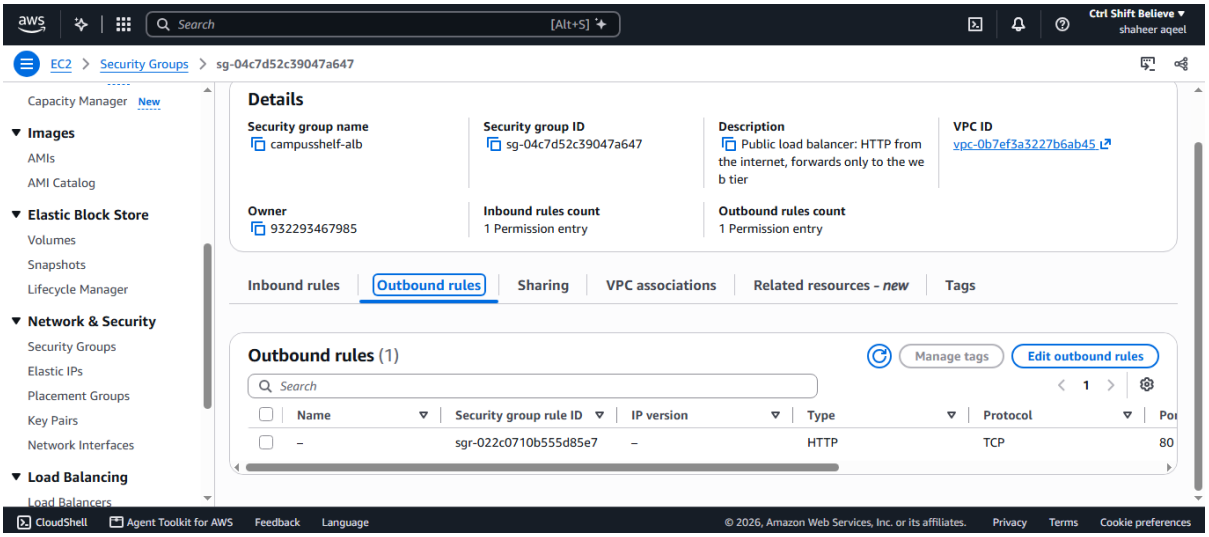


Figure 12: *ALB security group: outbound only to the web tier*

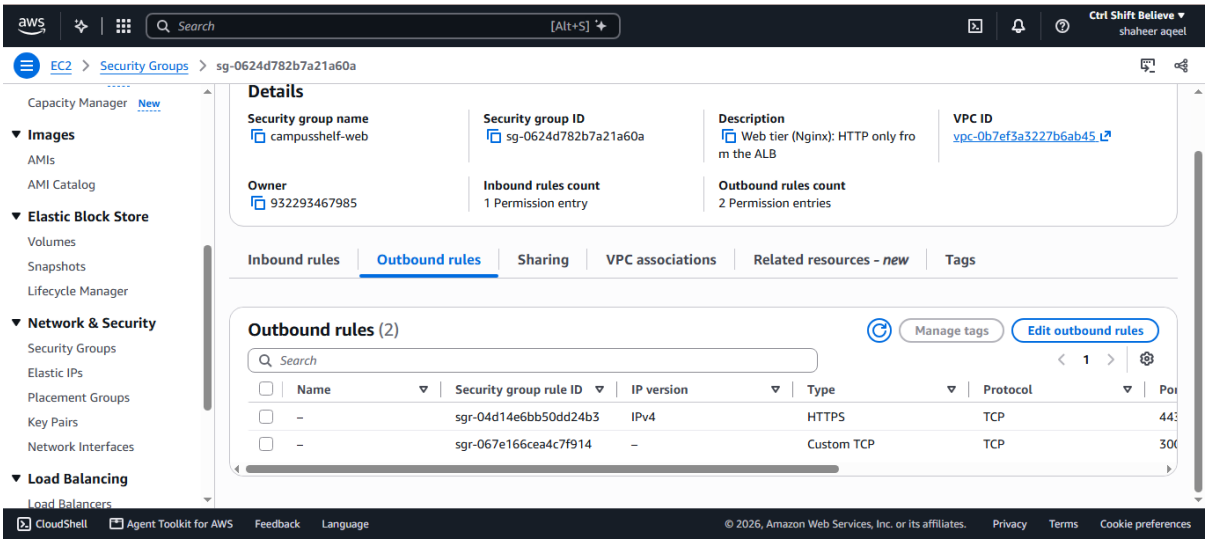


Figure 13: *Web tier (Nginx) security group: HTTP only from the ALB*

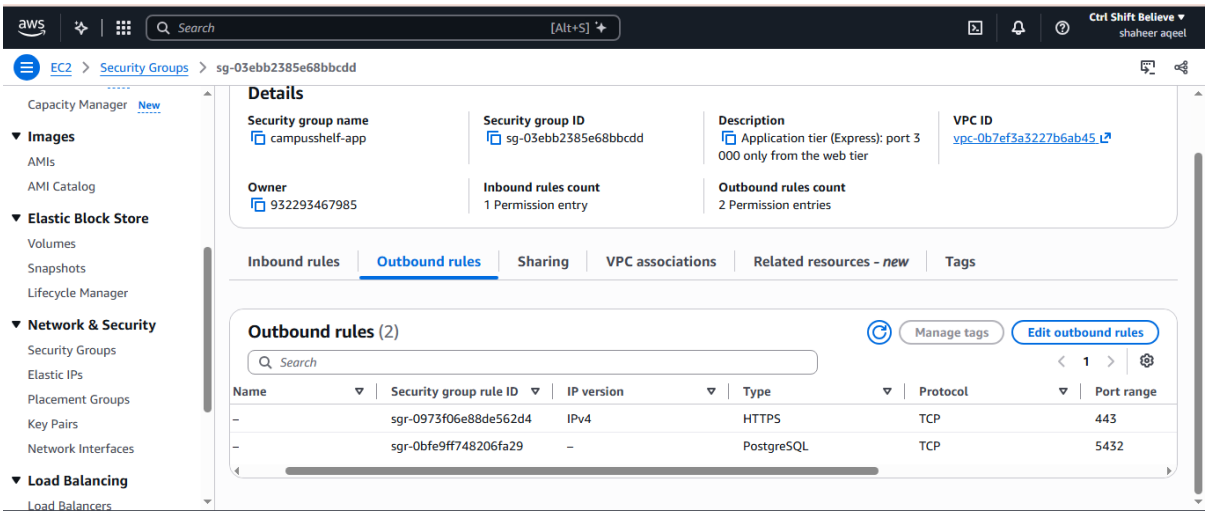


Figure 14: *App tier (Express) security group: inbound only from the web tier, outbound HTTPS and PostgreSQL*

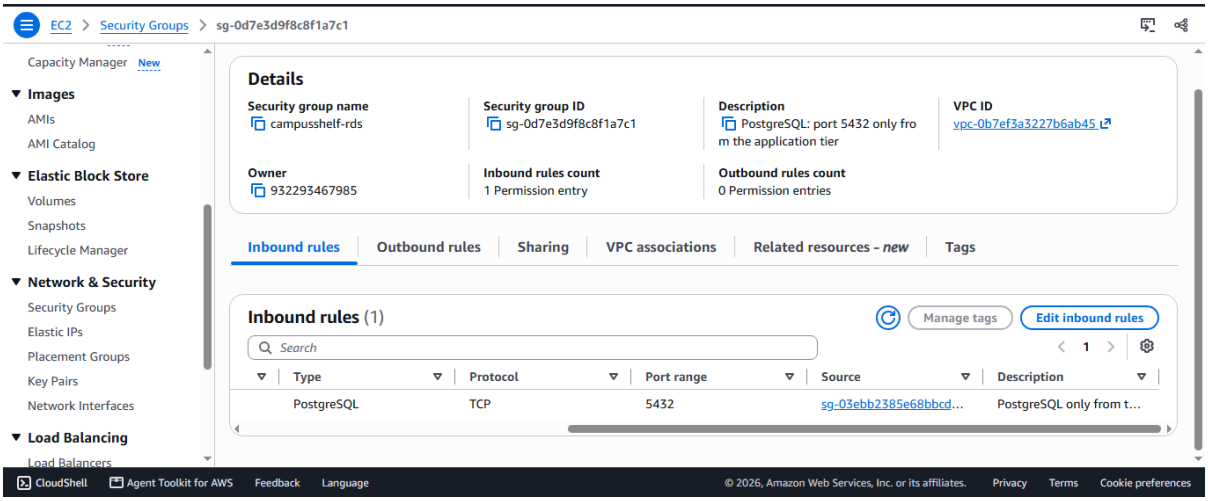


Figure 15: *RDS security group: PostgreSQL 5432 only from the app tier*

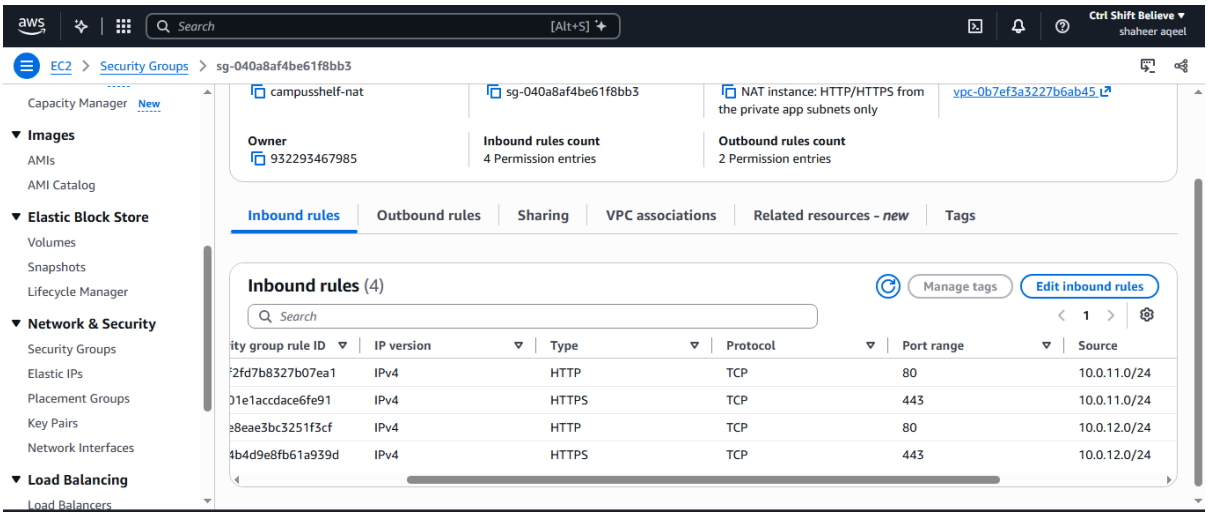


Figure 16: *NAT security group: inbound HTTP/HTTPS from the private app subnets*

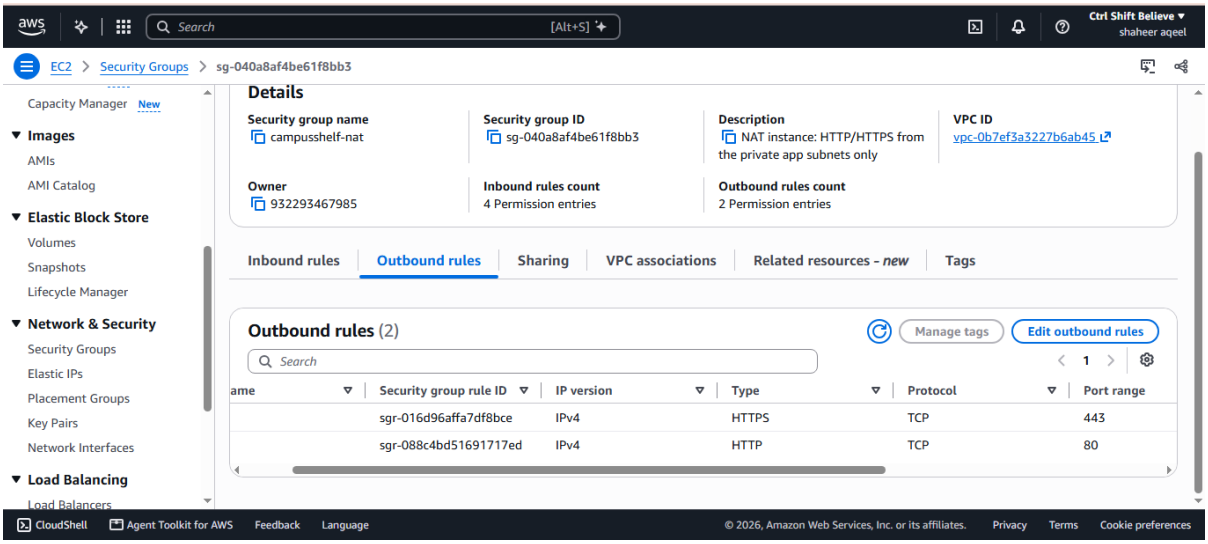


Figure 17: *NAT security group: outbound rules*

4. Load Balancer Configuration

An internet-facing Application Load Balancer in the two public subnets forwards HTTP traffic to the web tier target group. Targets are health-checked on /healthz.

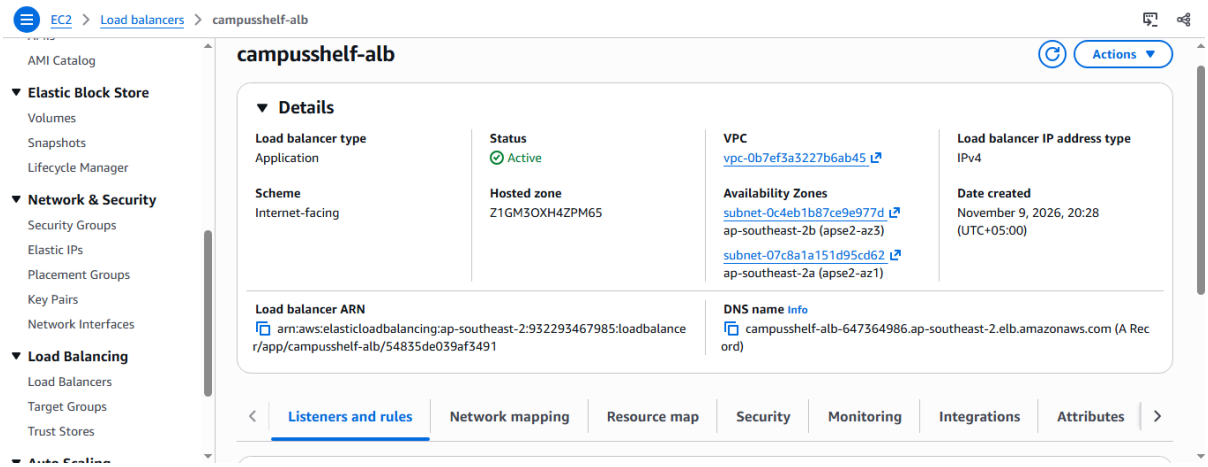


Figure 18: *Application Load Balancer campusshelf-alb: Active, internet-facing*

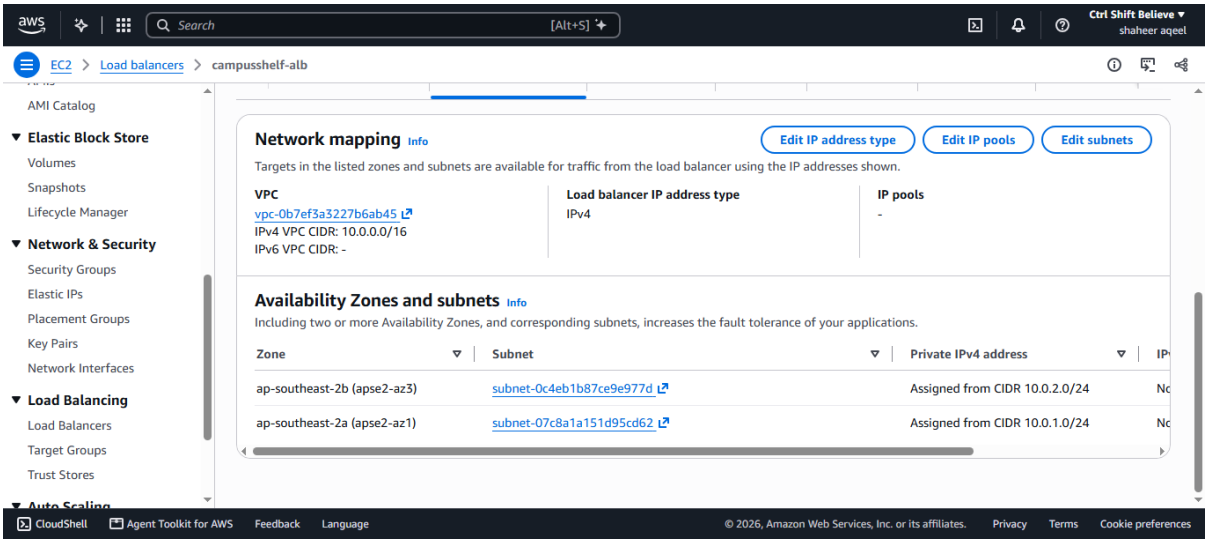


Figure 19: *ALB network mapping across two Availability Zones (public subnets)*

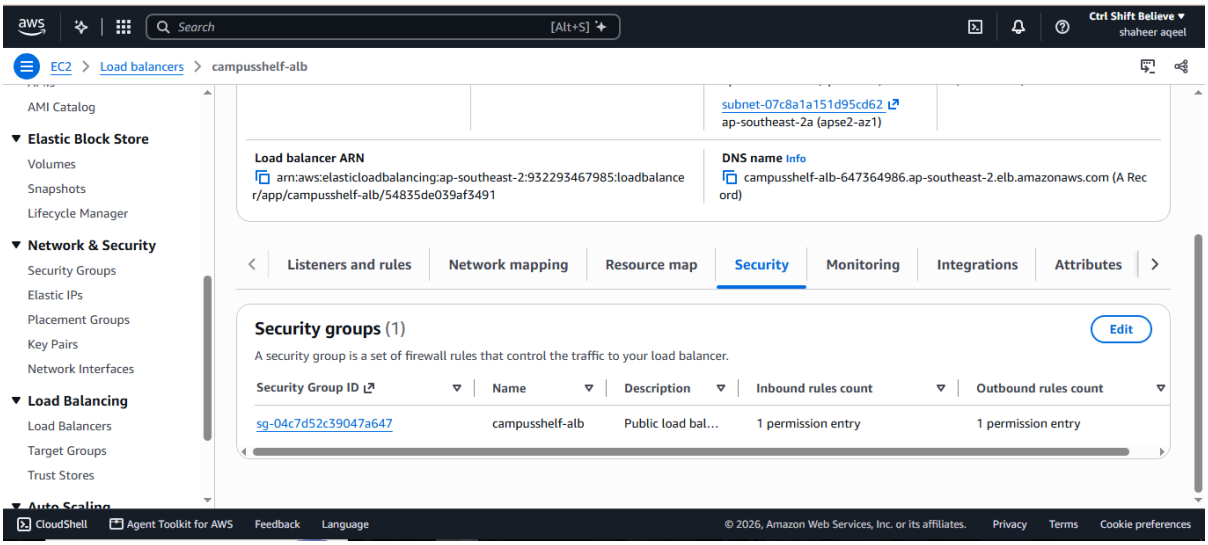


Figure 20: *ALB attached to its security group*

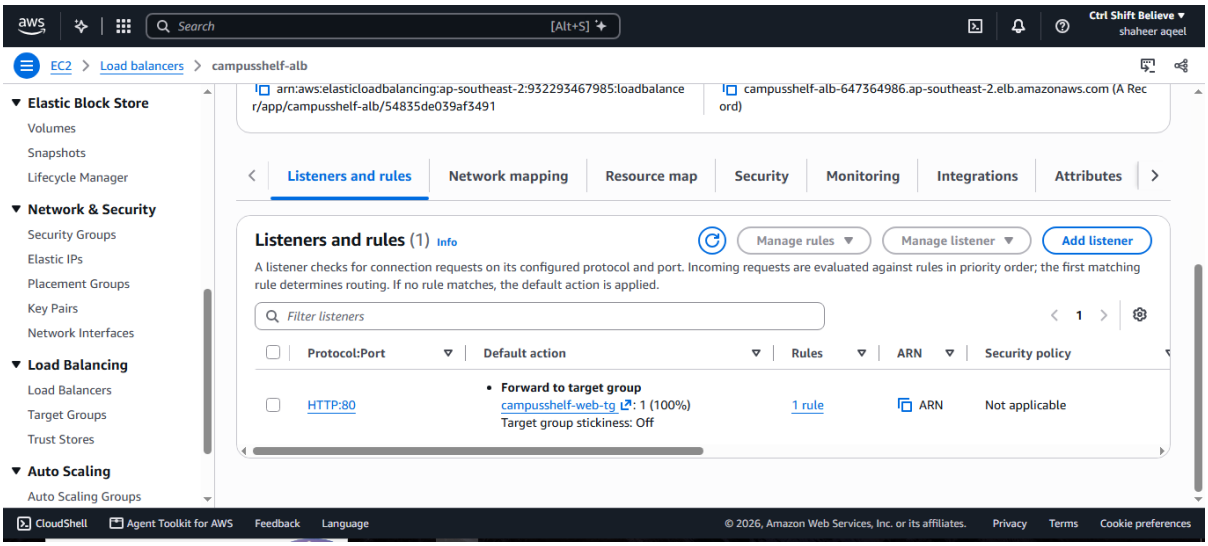


Figure 21: *Listener HTTP:80 forwarding 100% of traffic to campusshelf-web-tg*

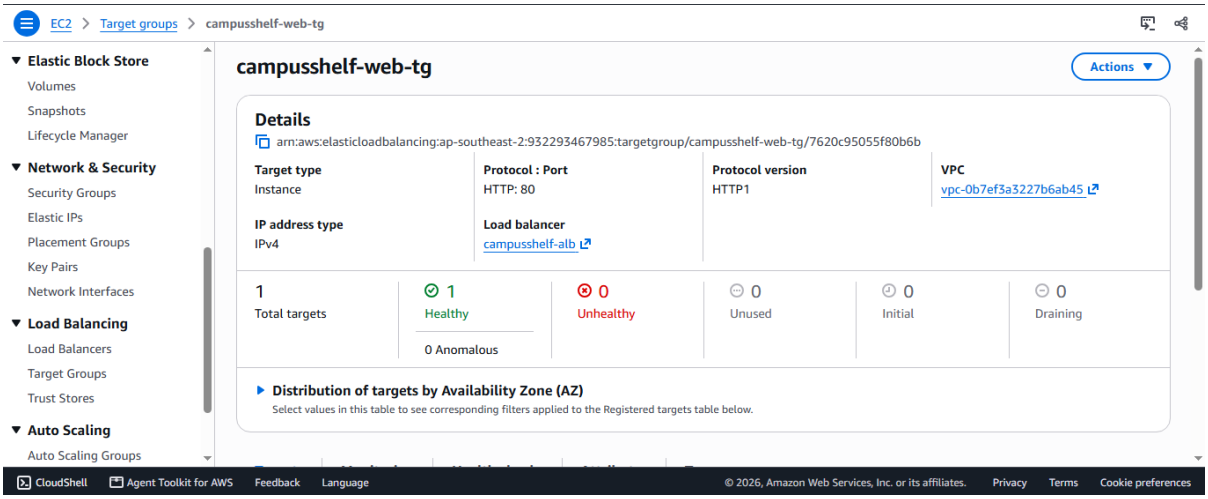


Figure 22: *Target group campusshelf-web-tg details*

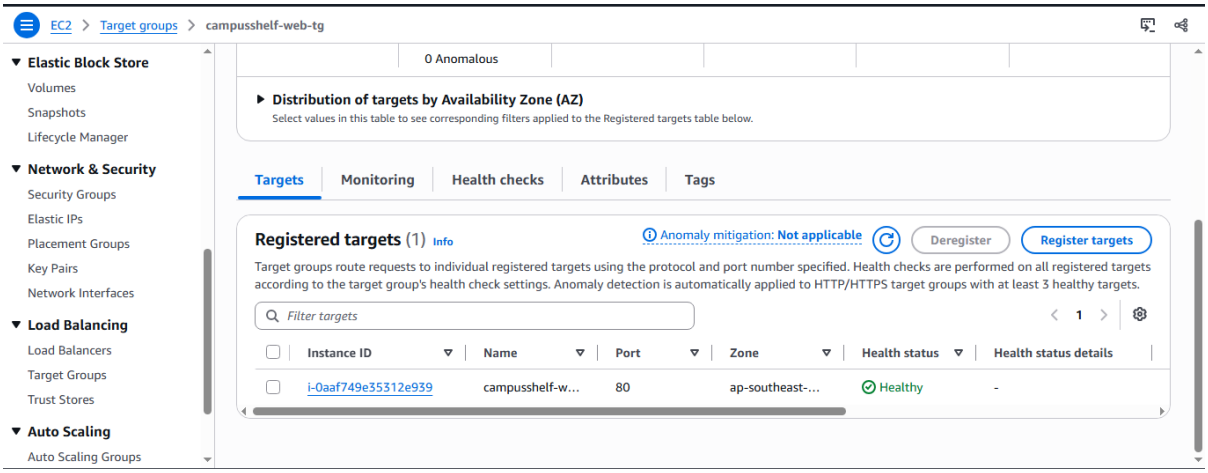


Figure 23: *Registered web target reported Healthy*

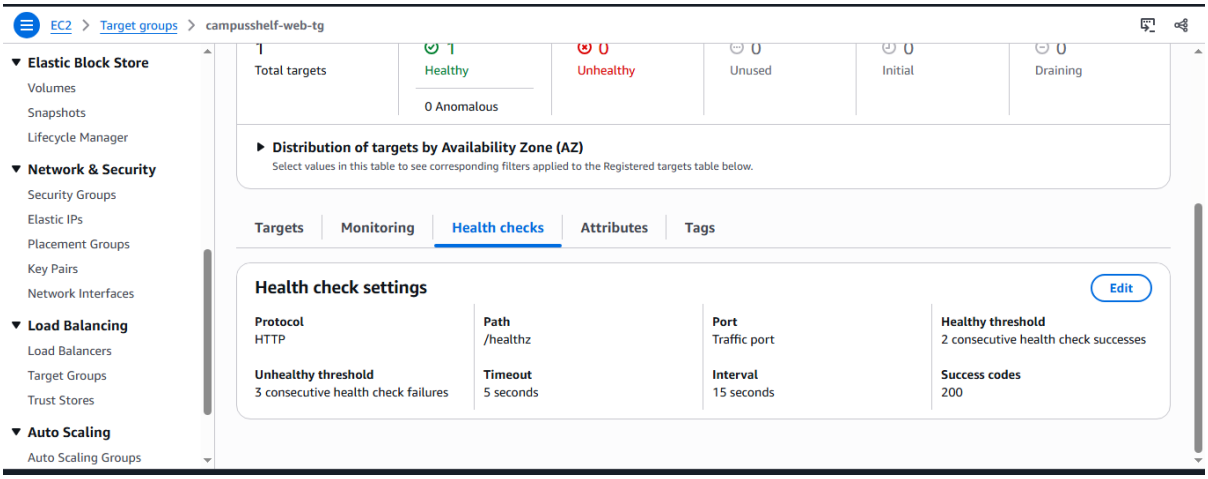


Figure 24: *Health check settings: HTTP /healthz, success code 200*

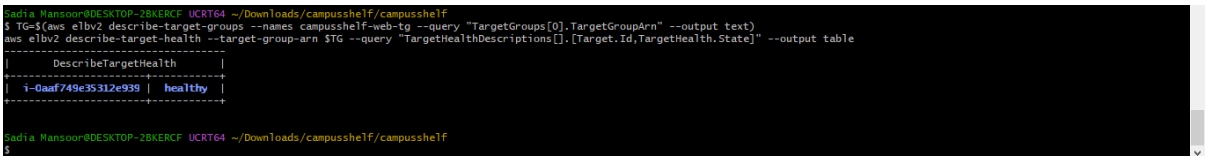


Figure 25: *Target health checked from the CLI: Healthy*

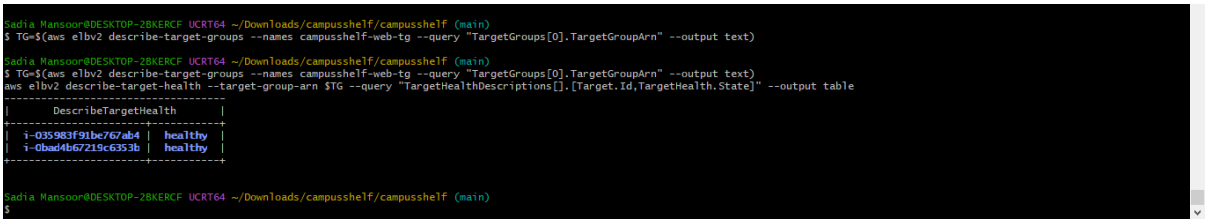


Figure 26: *Both web targets Healthy (CLI)*

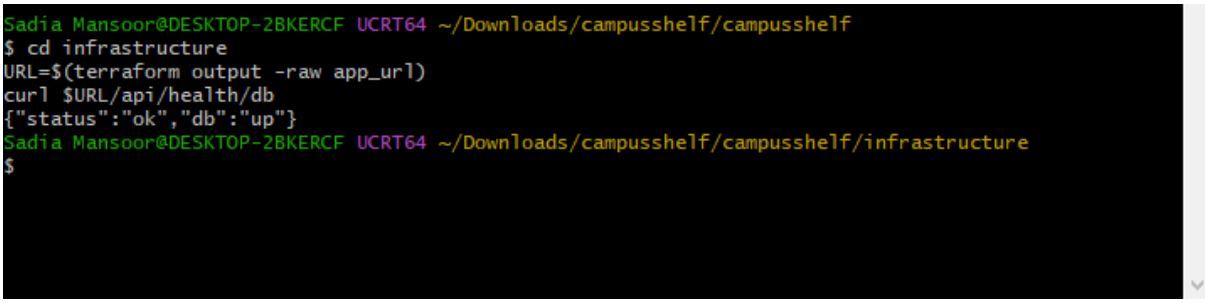


Figure 27: *End-to-end database health check through the ALB: {"status":"ok","db":"up"}*

5. Database Configuration

PostgreSQL 16 runs on Amazon RDS (db.t4g.micro, single-AZ, 20 GB gp2, inside the free tier) in the private database subnets. It is not publicly accessible.

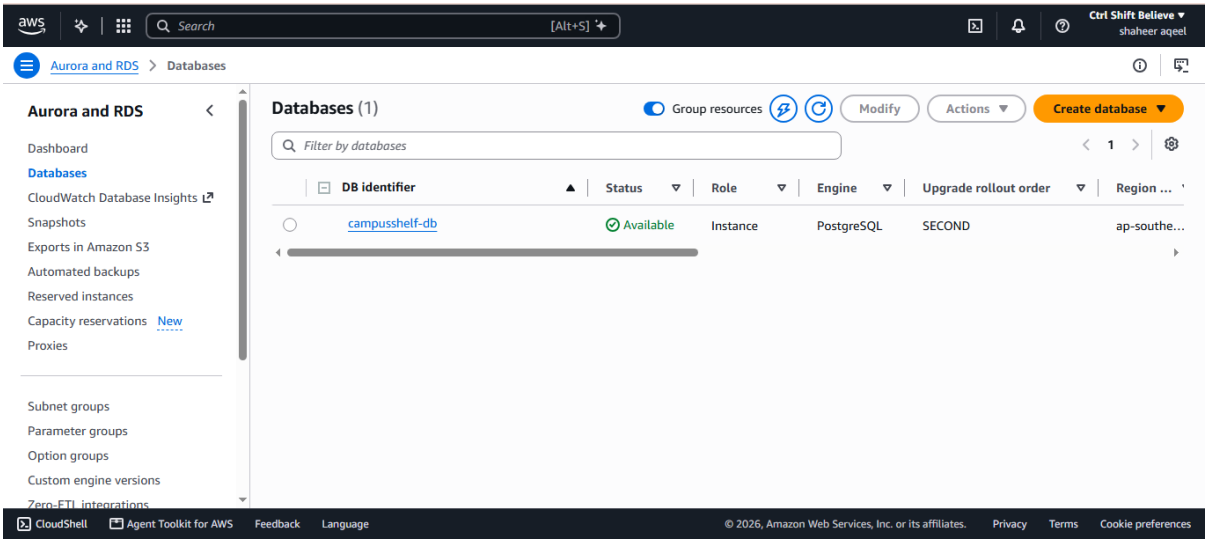


Figure 28: *RDS PostgreSQL instance campusshelf-db: Available*

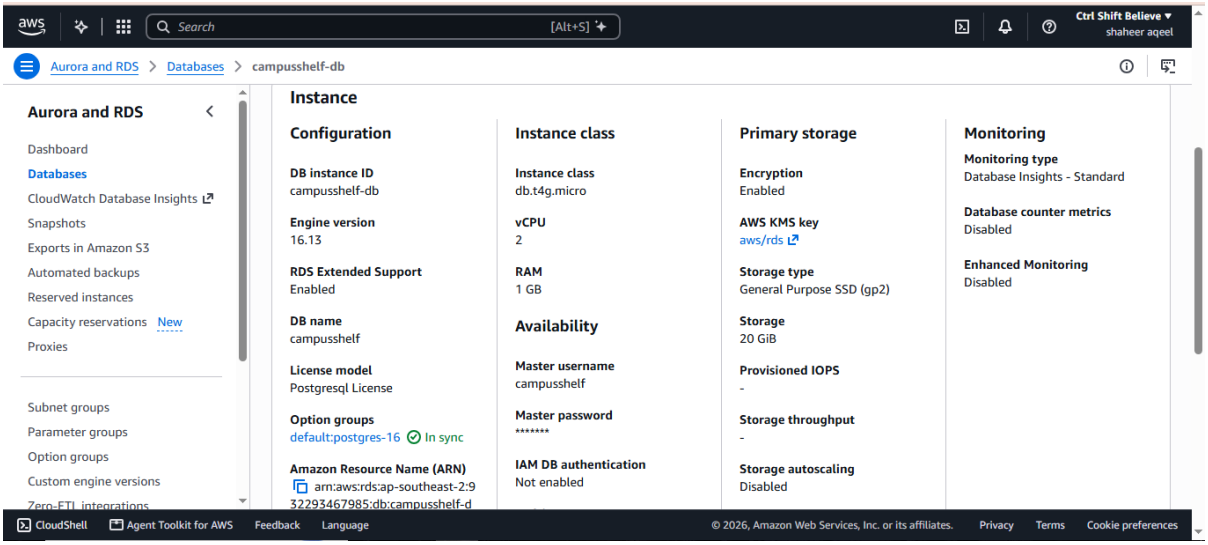


Figure 29: *RDS configuration: PostgreSQL 16, db.t4g.micro, 20 GB storage*

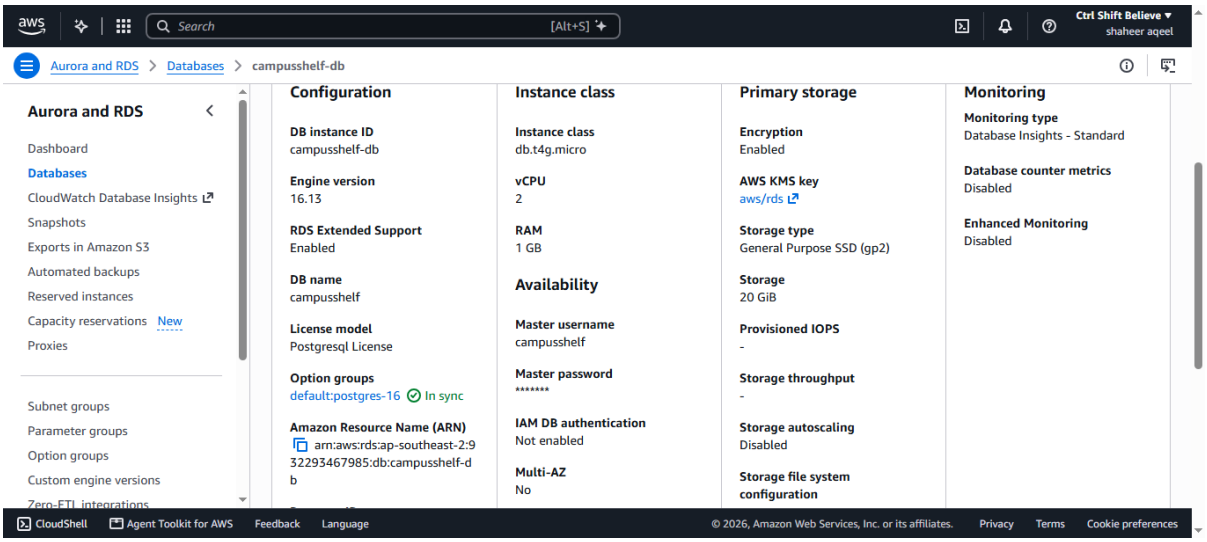


Figure 30: *RDS storage gp2, encryption enabled, Multi-AZ off (free tier)*

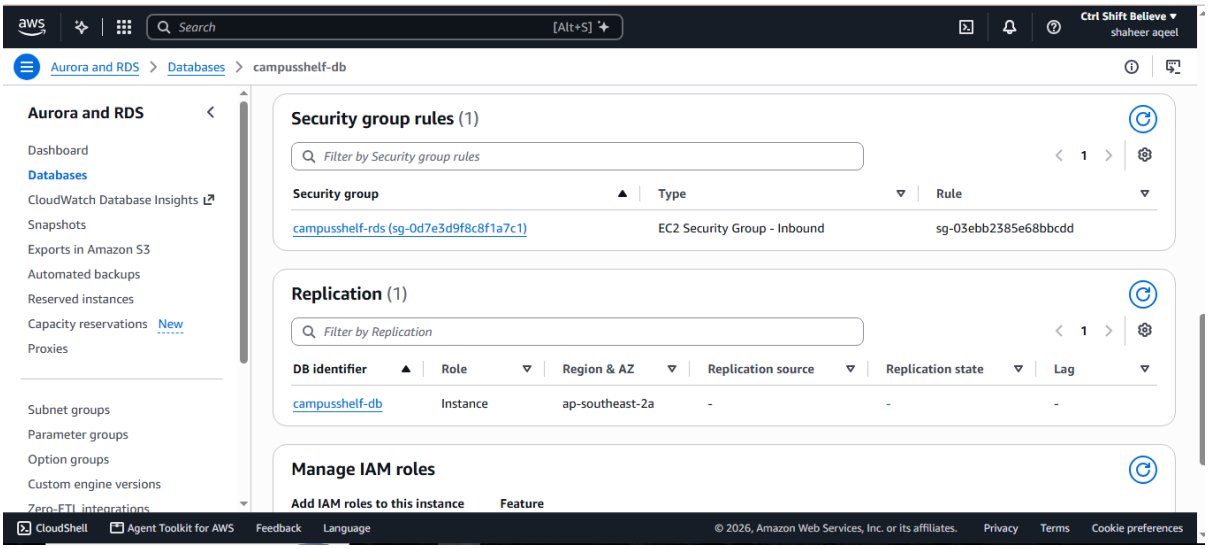


Figure 31: *RDS security group rule (inbound only from the app tier)*

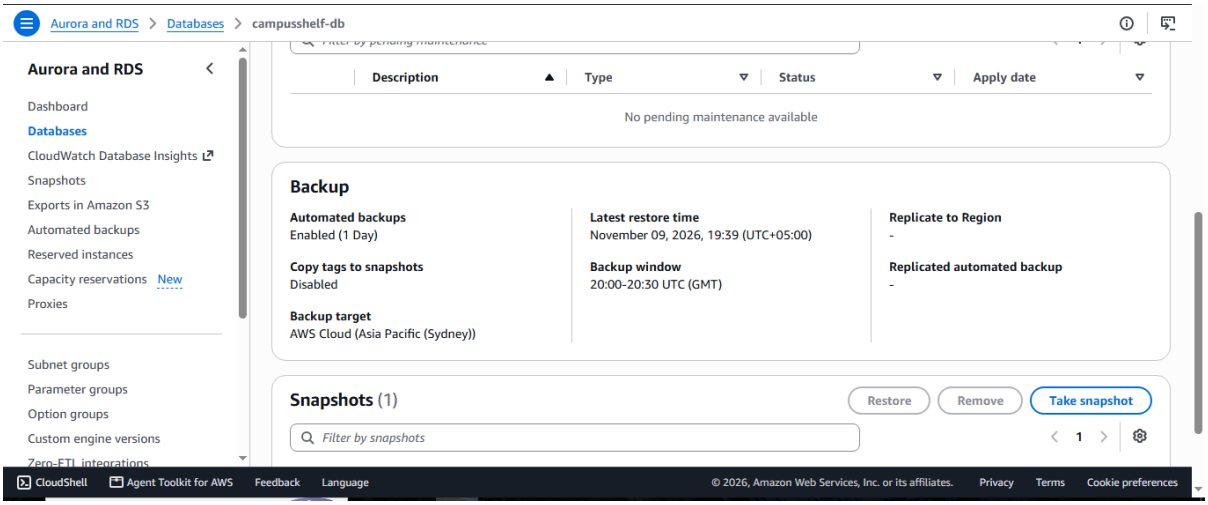


Figure 32: *Automated backups enabled (1-day retention)*

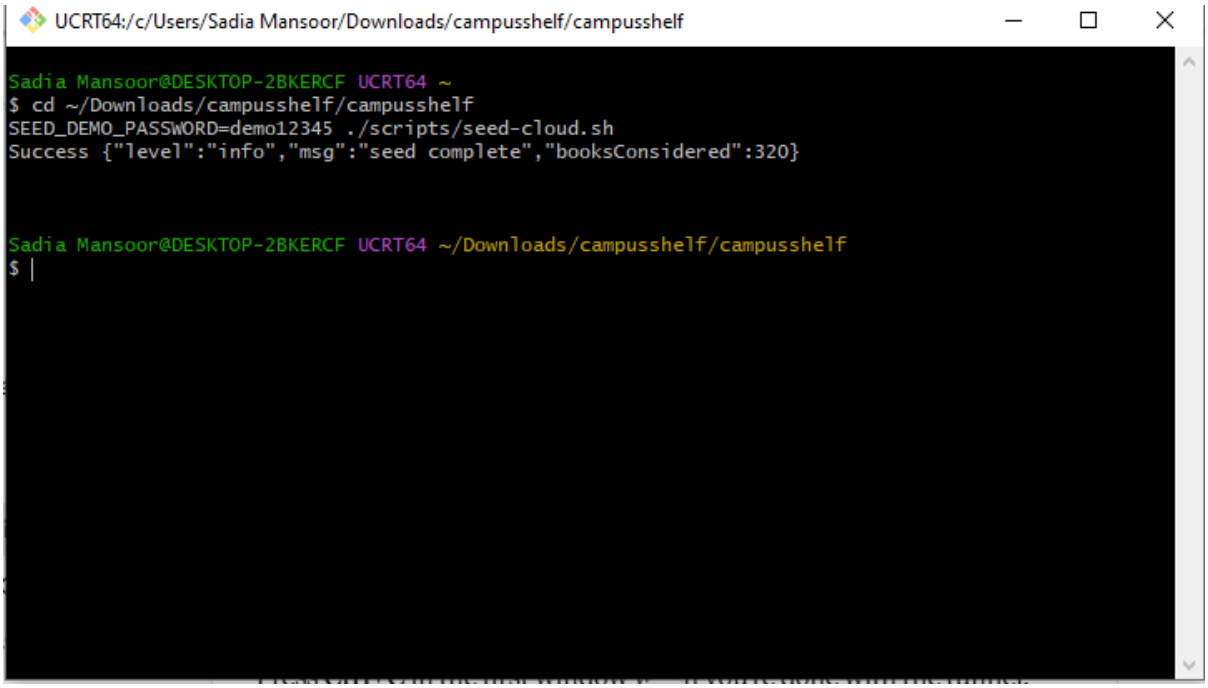


Figure 33: *Database seeded on AWS: 320 books loaded*

6. Docker Build and Registry (Amazon ECR)

The web (React + Nginx) and api (Node.js + Express) images are built with multi-stage Dockerfiles, tested locally with Docker Compose, then pushed to Amazon ECR repositories with immutable tags.

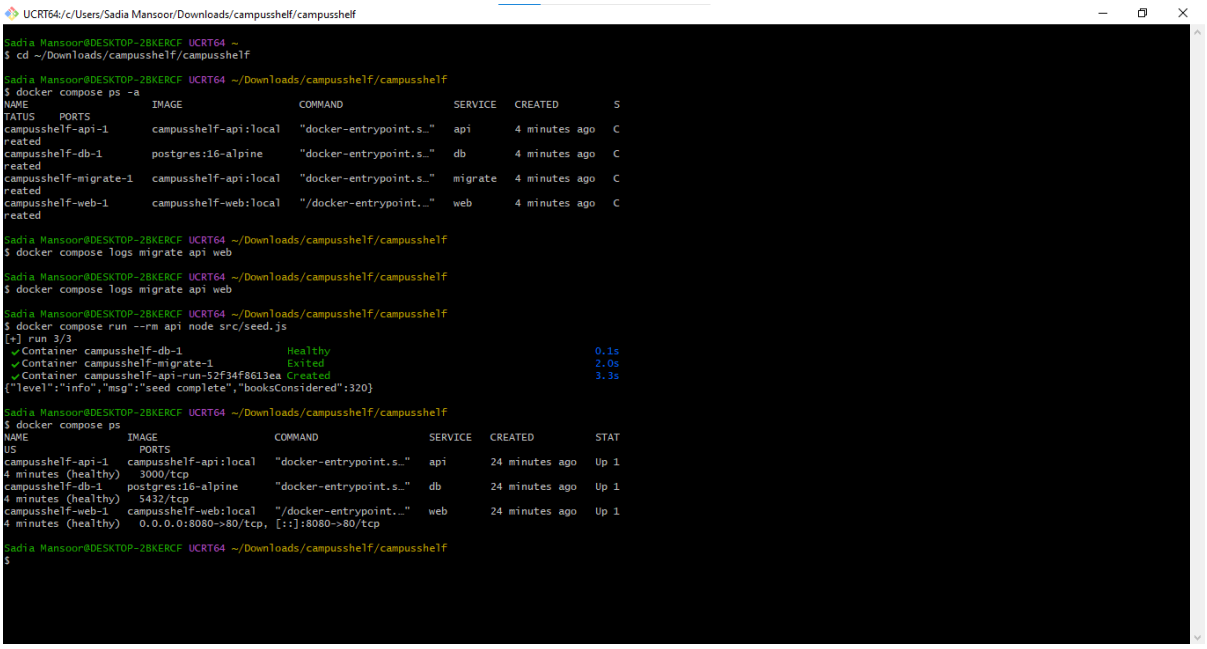


Figure 34: *Local Docker Compose stack: api, db and web containers healthy, database seeded*

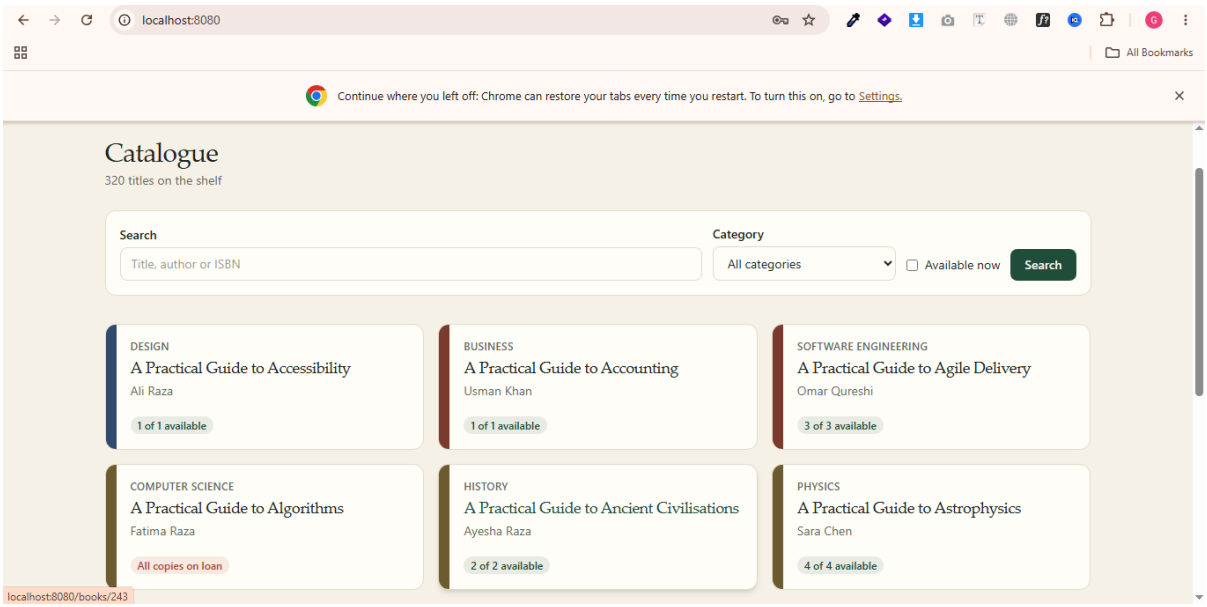


Figure 35: *Application running locally in containers (localhost:8080)*

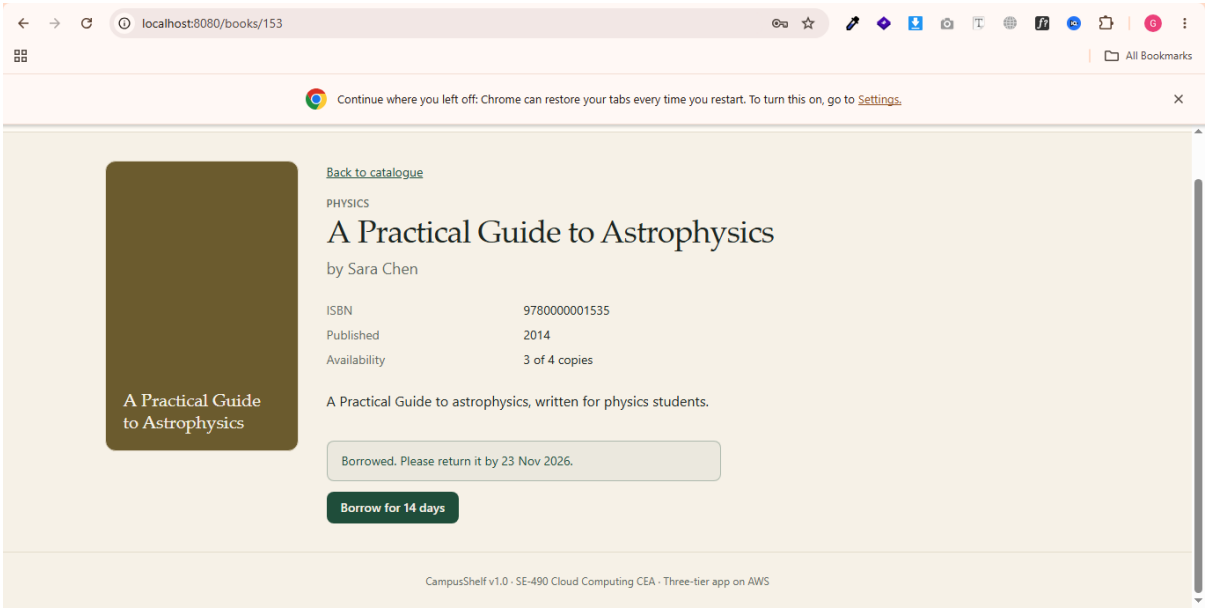


Figure 36: *Borrowing a book in the local containerised application*

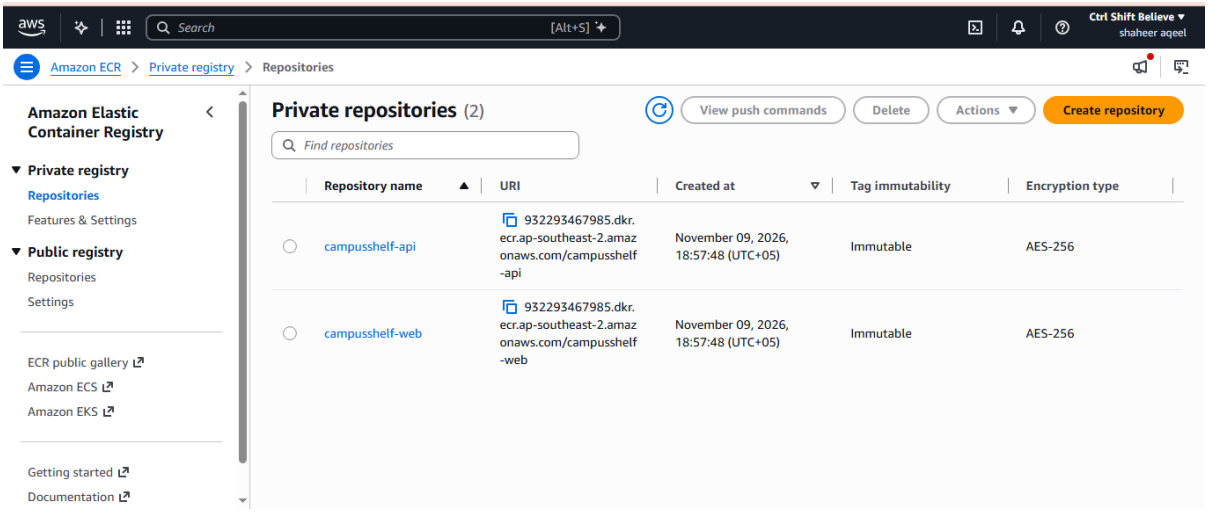


Figure 37: *Private ECR repositories campusshelf-api and campusshelf-web (tag immutability on)*

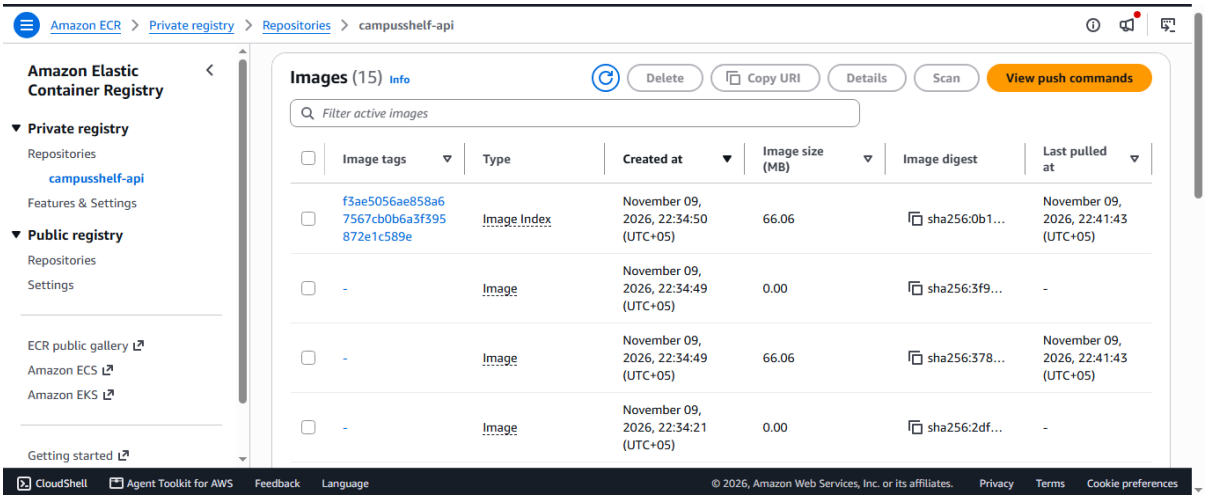


Figure 38: *Images in campusshelf-api tagged with the Git commit SHA*

7. CI/CD Pipeline Execution (GitHub Actions)

The workflow .github/workflows/ci-cd.yml runs the API and web tests, builds the Docker images, pushes them to ECR, waits for manual approval of the production environment, then rolls them out with ASG instance refreshes and a health check.

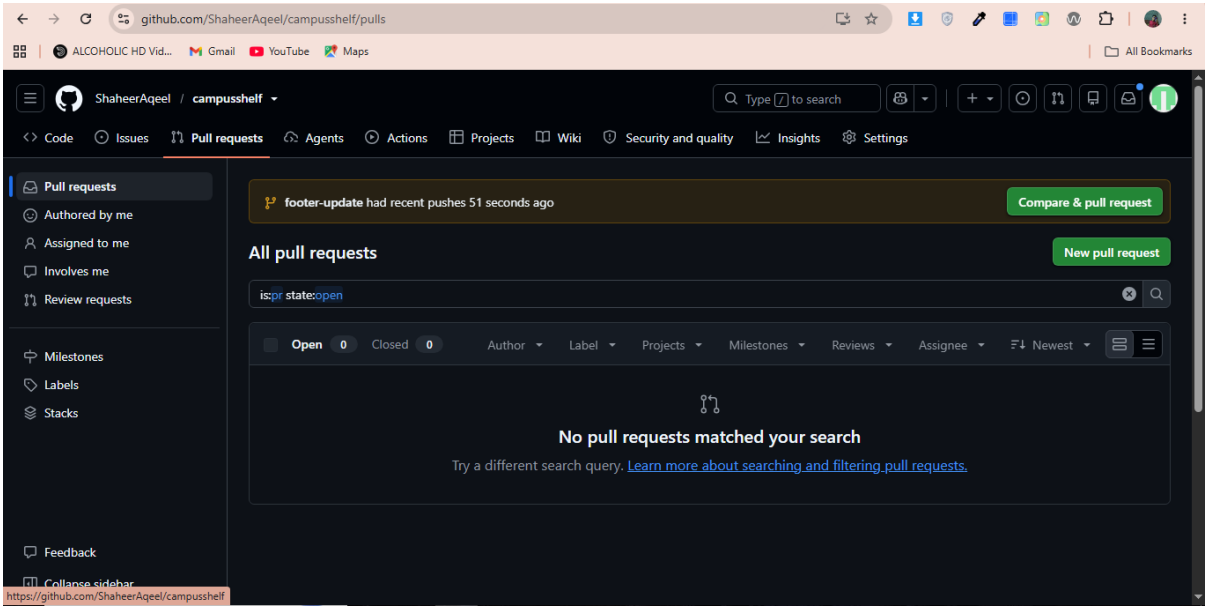


Figure 39: *Pull request for the footer v1.1 change*

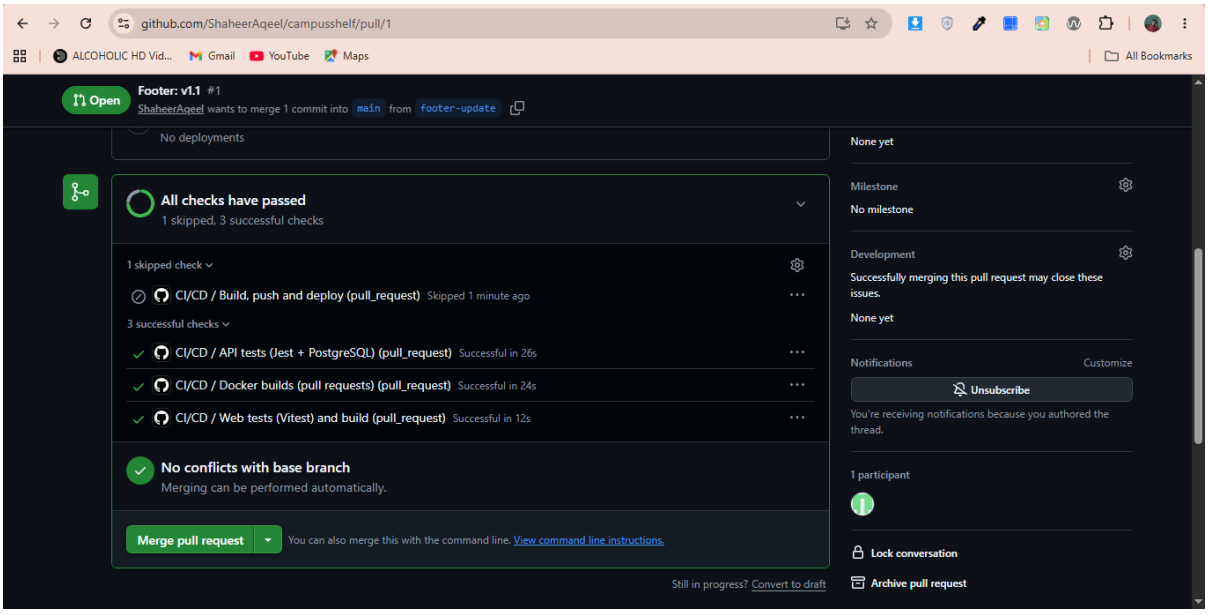


Figure 40: *Pull request checks passed: API tests, web tests and Docker builds*

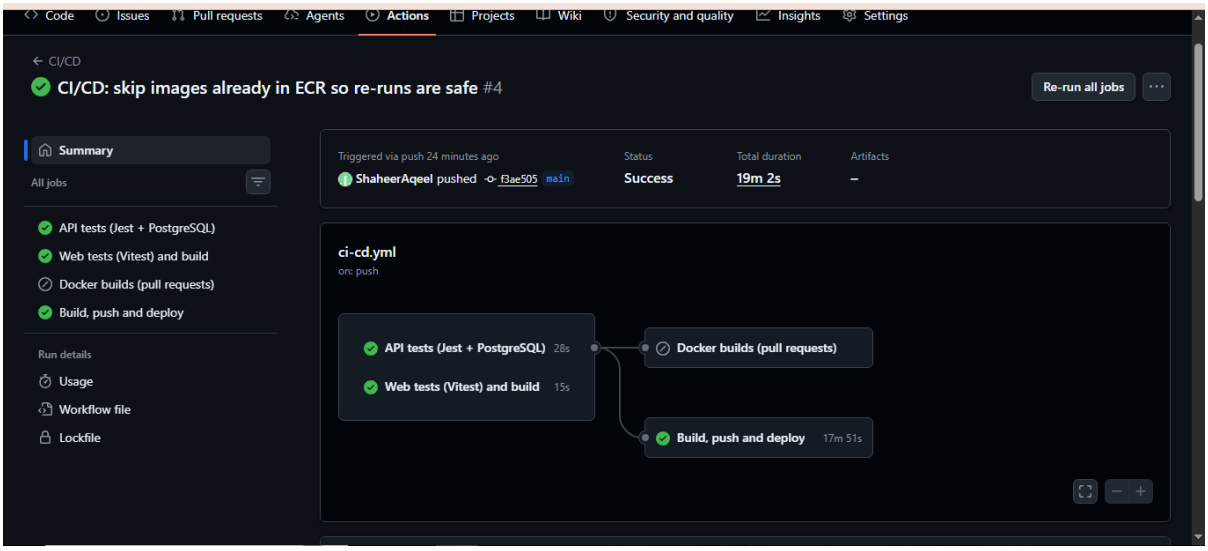


Figure 41: *Successful pipeline run: tests, then Build, push and deploy*

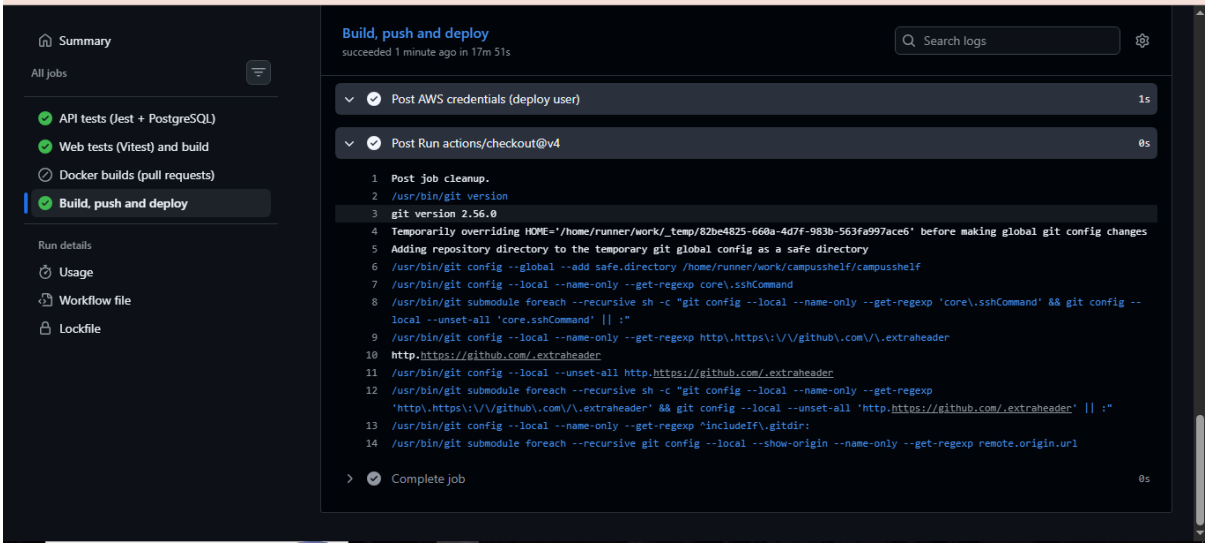


Figure 42: *Build, push and deploy job log*

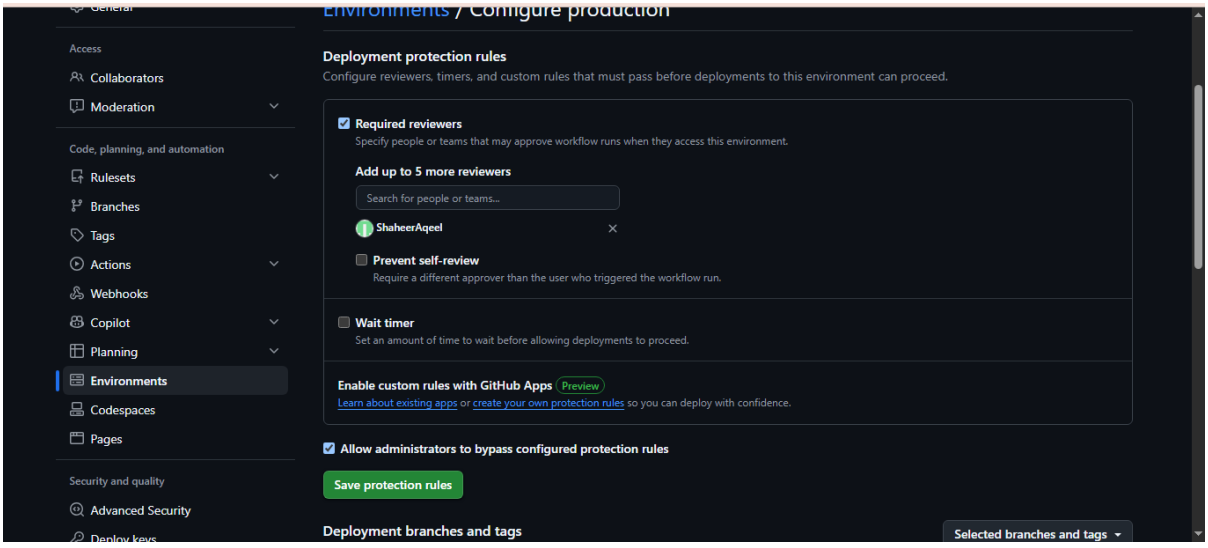


Figure 43: *Production environment protection rule: required reviewer*

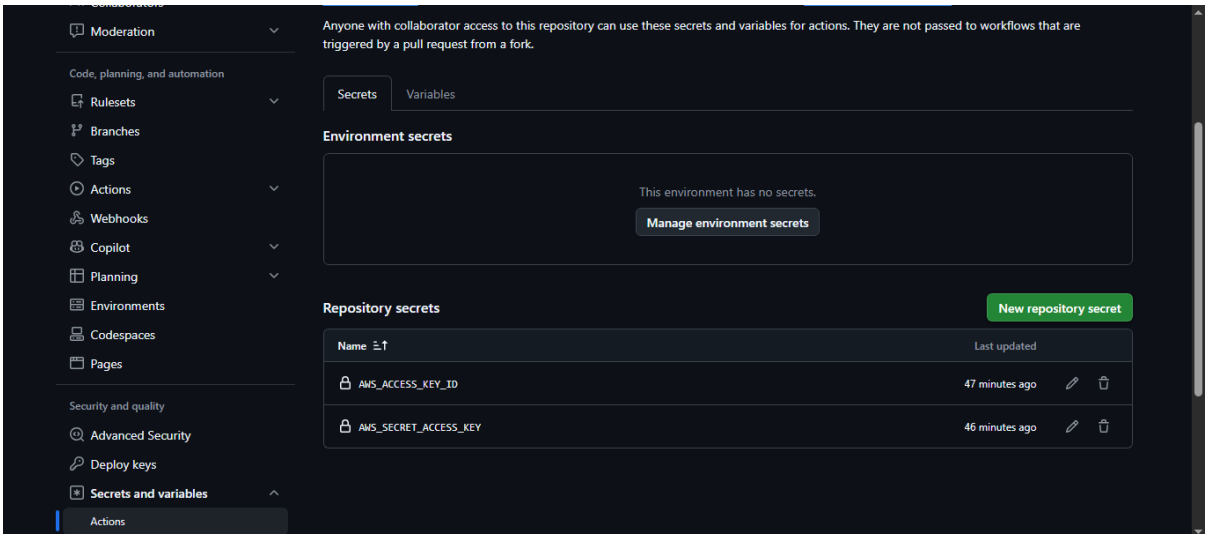


Figure 44: *Repository secrets for the IAM deploy user*

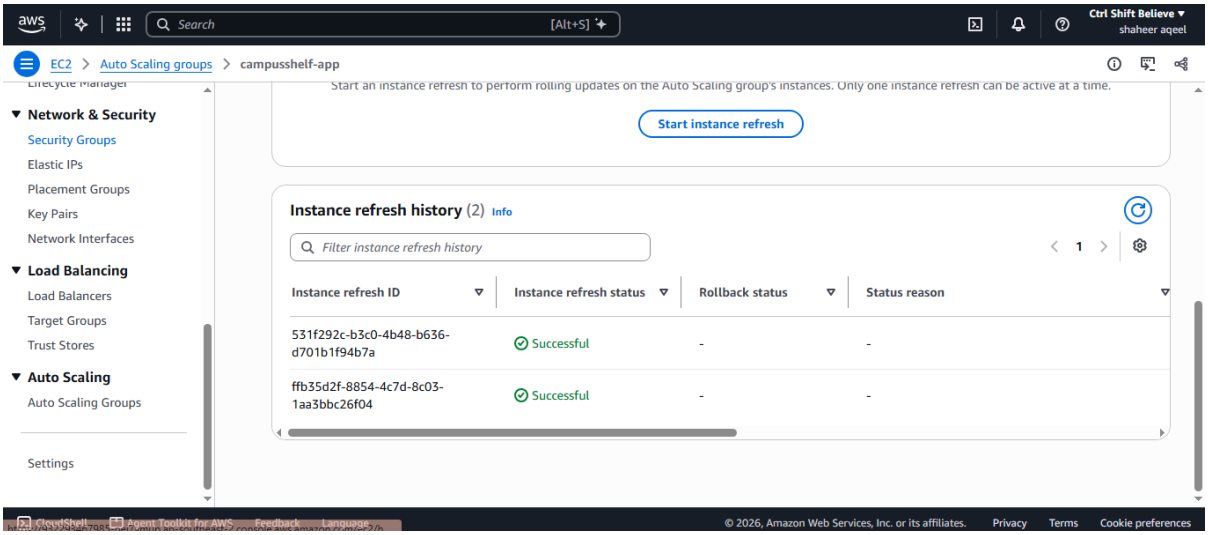


Figure 45: *Instance refresh history: rolling deployments Successful*

8. Running Application in Browser

The deployed application is reachable through the ALB DNS name. The web tier serves the React frontend, and the data comes from the api tier and PostgreSQL on RDS.

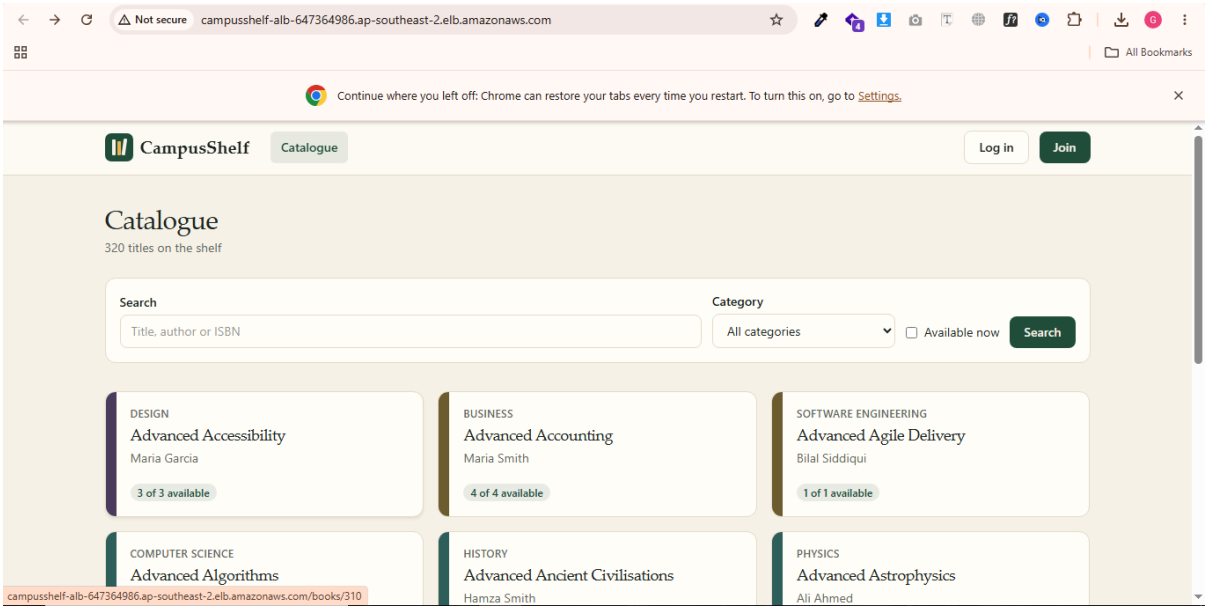


Figure 46: *CampusShelf catalogue served through the ALB (320 titles from RDS)*

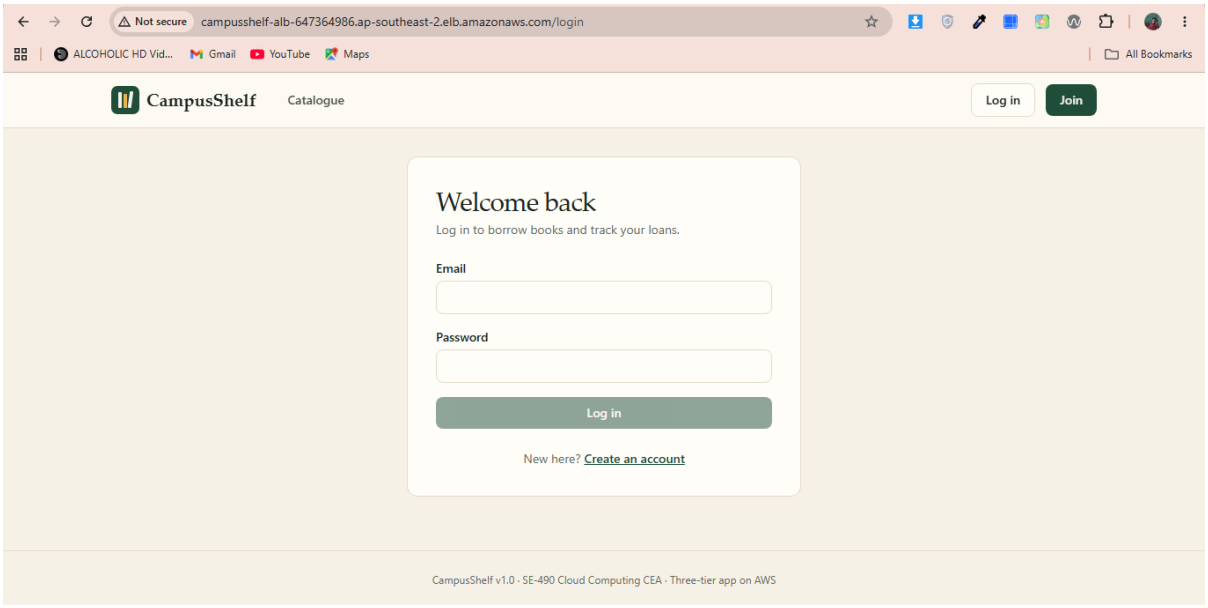


Figure 47: *Login page of the deployed application*

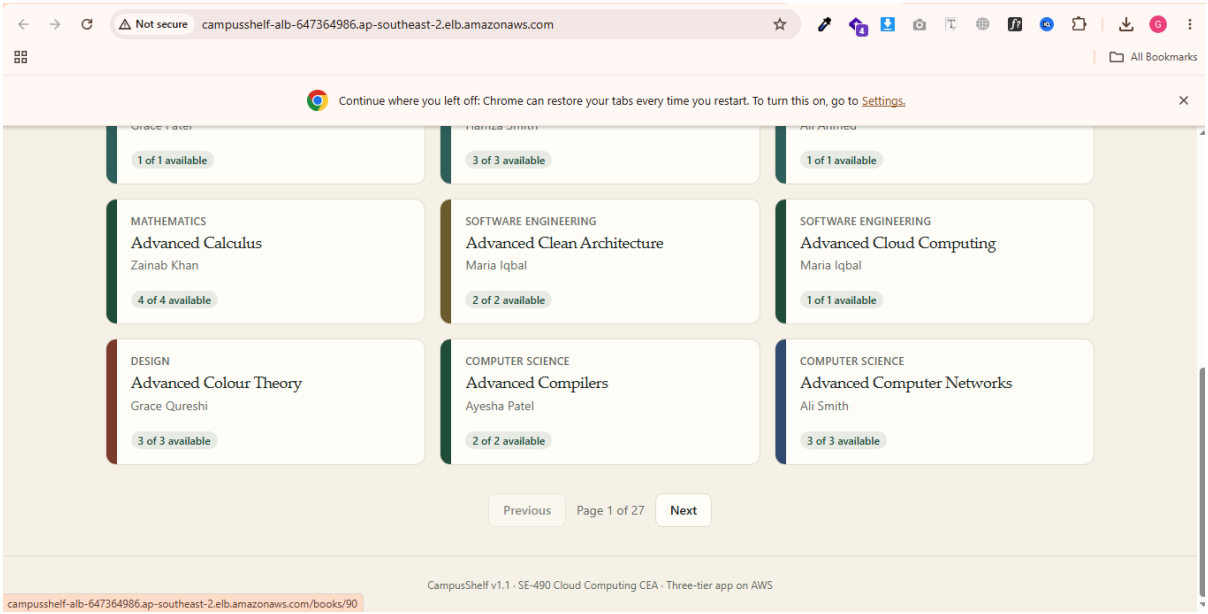


Figure 48: *Version 1.1 live after the CI/CD deployment (updated footer)*

9. Infrastructure as Code (Terraform)

All AWS resources are provisioned with Terraform from the infrastructure/ folder.

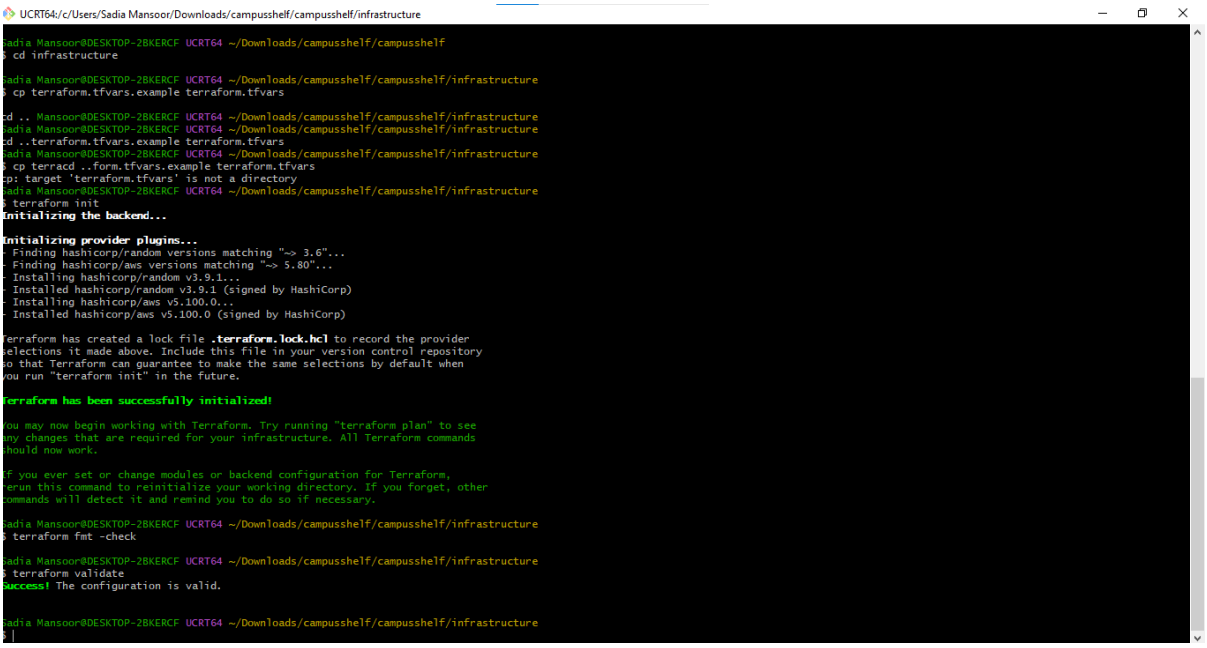


Figure 49: *terraform init, fmt -check and validate: configuration valid*

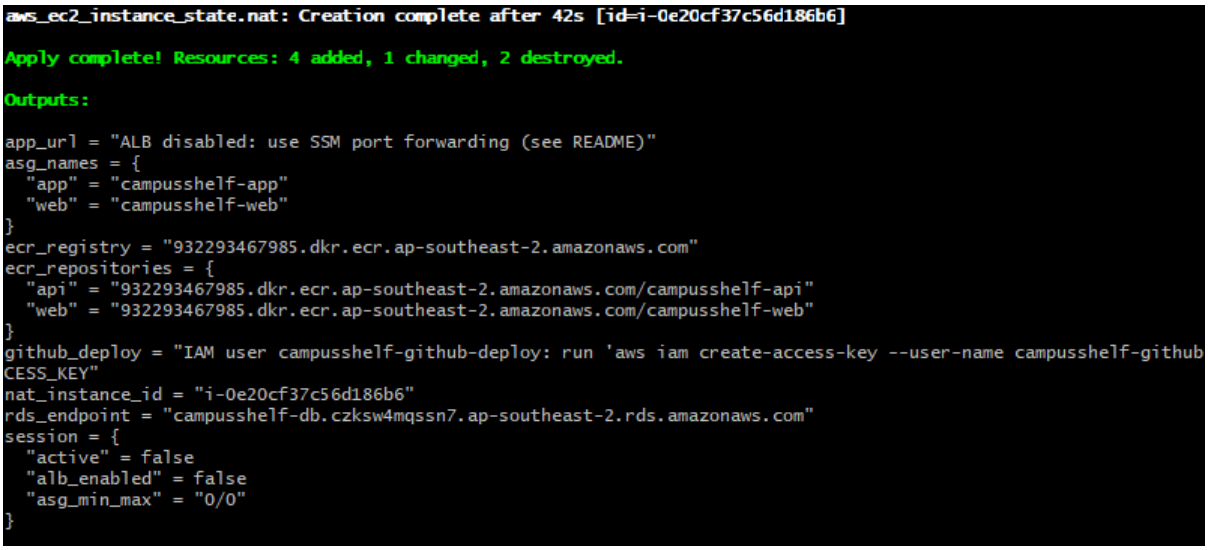


Figure 50: *terraform apply complete with outputs*



Figure 51: *Terraform outputs: ALB URL, ECR repositories, RDS endpoint*

10. Secrets and Access Management

The database password and JWT secret are SecureString parameters in SSM Parameter Store. The EC2 instances use IAM roles, and the pipeline uses a least-privilege IAM deploy user.

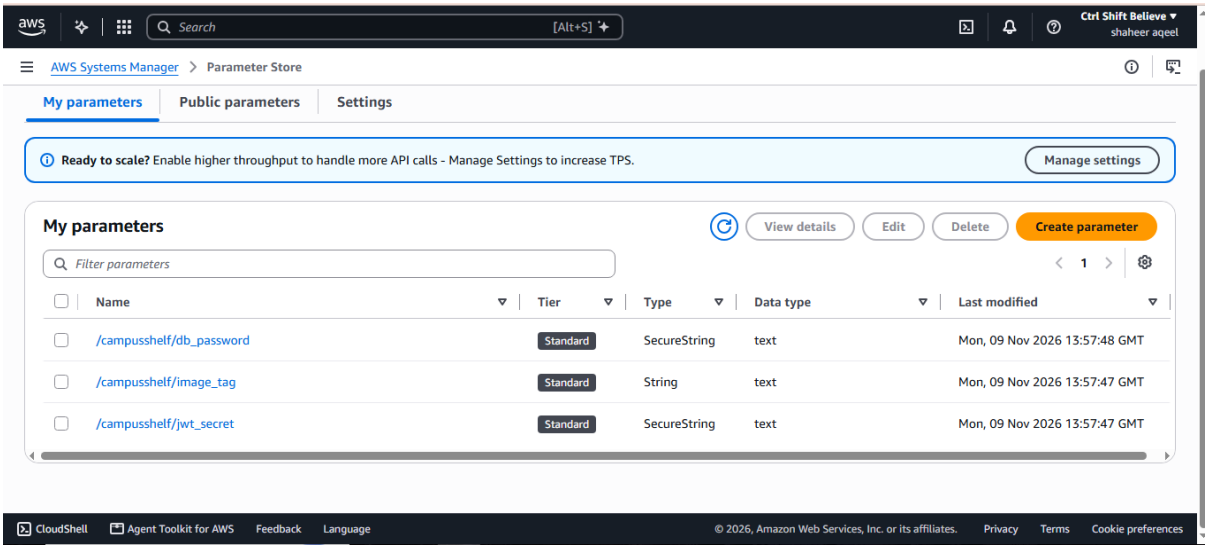


Figure 52: *SSM Parameter Store: db_password and jwt_secret (SecureString), image_tag*

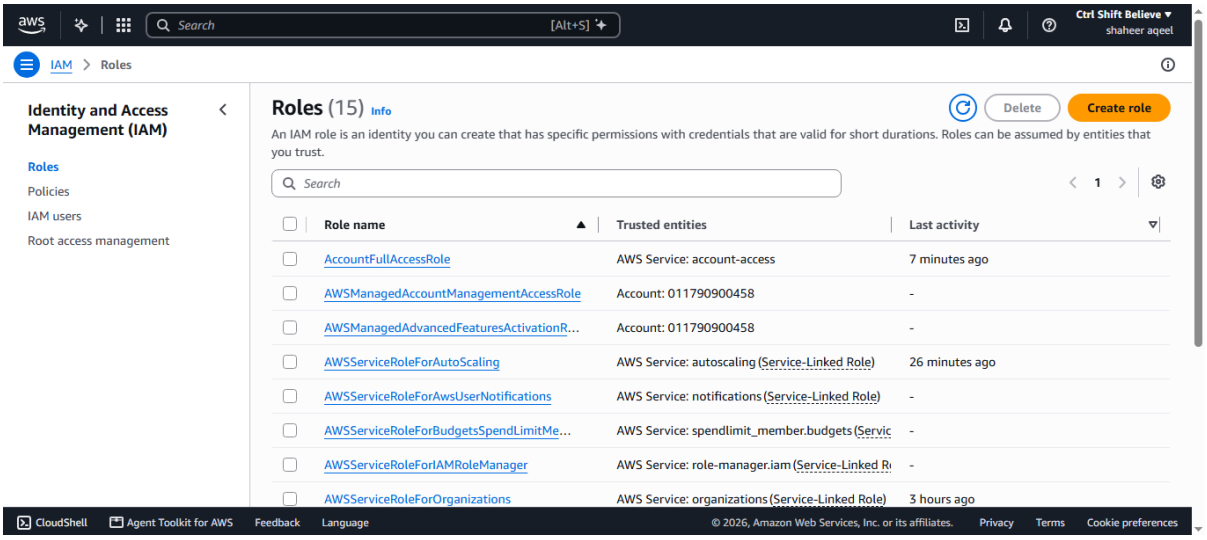


Figure 53: *IAM roles, including the service-linked role for Auto Scaling*

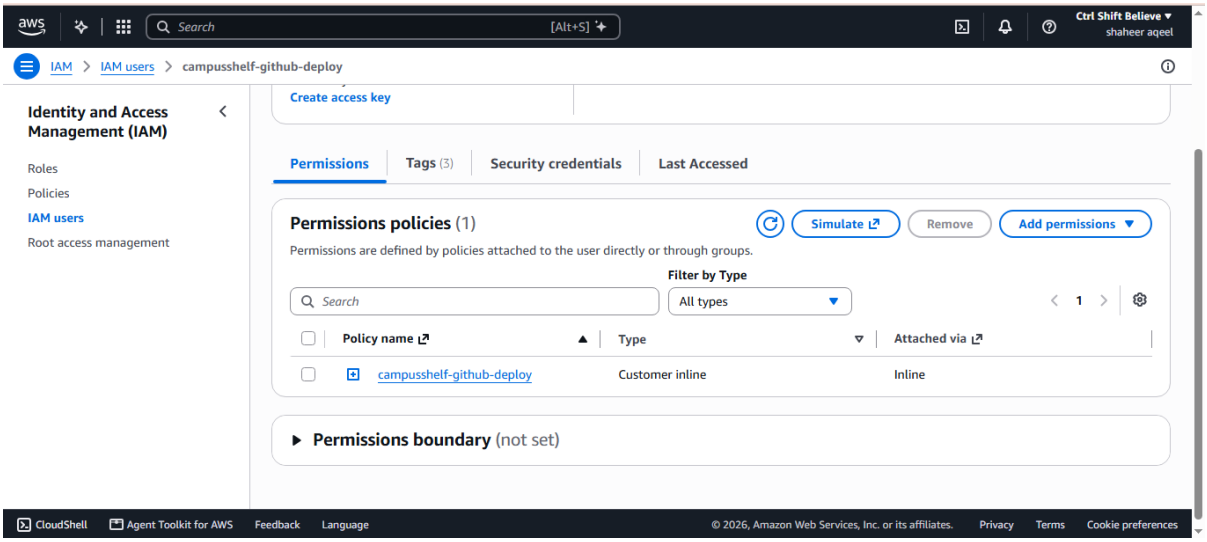


Figure 54: *IAM user campusshelf-github-deploy with an inline least-privilege policy*

11. Monitoring, Alarms and Cost Control

Container logs go to CloudWatch Logs. CloudWatch alarms drive scaling and send notifications through SNS, and an AWS Budget guards against charges.

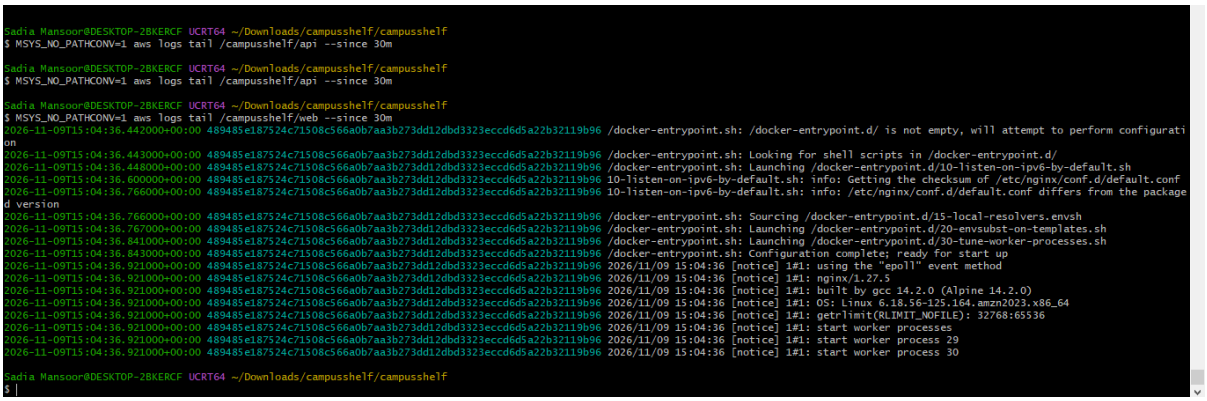


Figure 55: *CloudWatch Logs: web and api container logs*

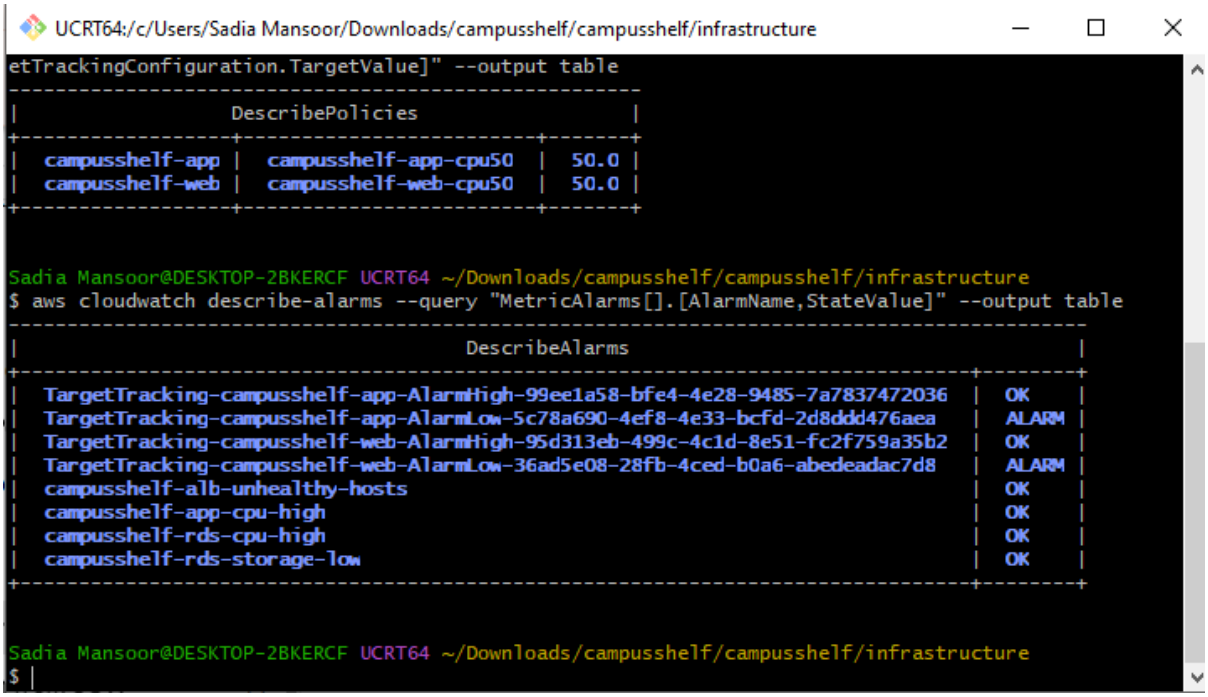


Figure 56: *CloudWatch alarms: target tracking, ALB unhealthy hosts, CPU and RDS storage*

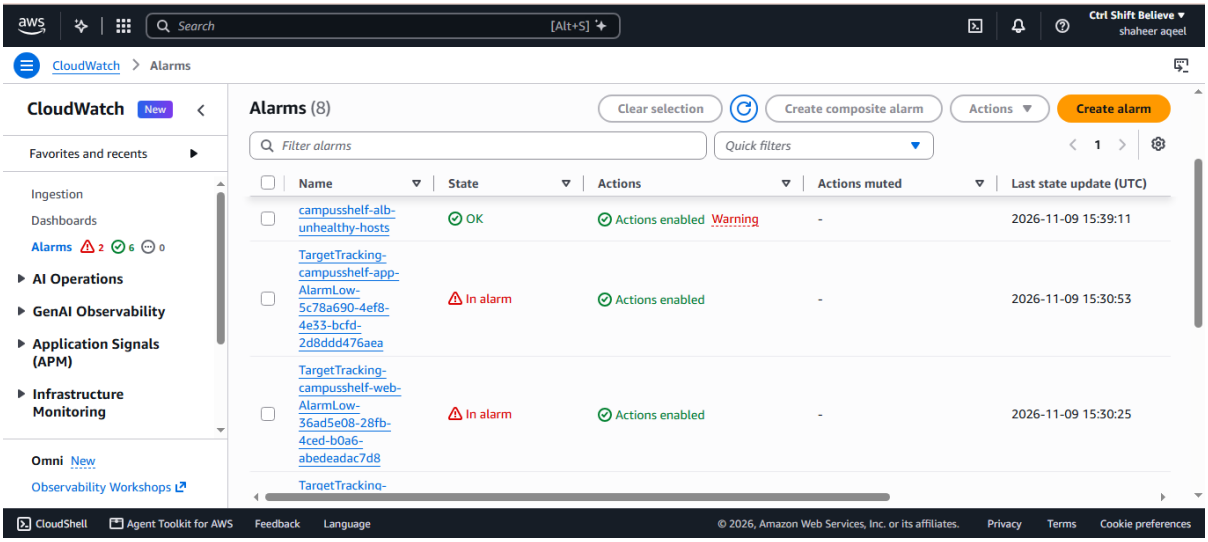


Figure 57: *CloudWatch alarms in the console*

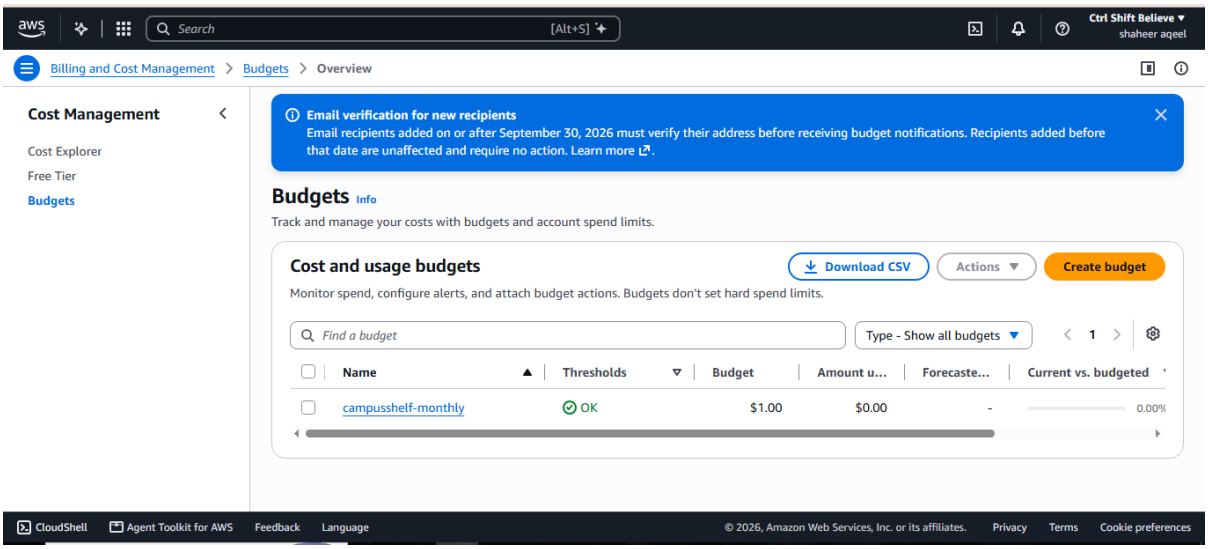


Figure 58: *AWS Budget campusshelf-monthly: $1.00 limit, $0.00 spent*

12. Scalability and Reliability Testing

Target-tracking scaling policies, autocannon load tests and a failure test (stopping a web instance) show that the system scales and heals itself.

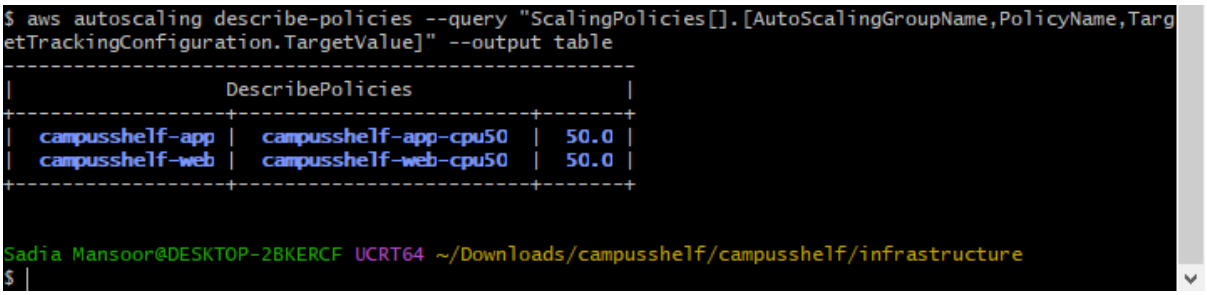


Figure 59: *Target-tracking scaling policies: average CPU 50% for both tiers*

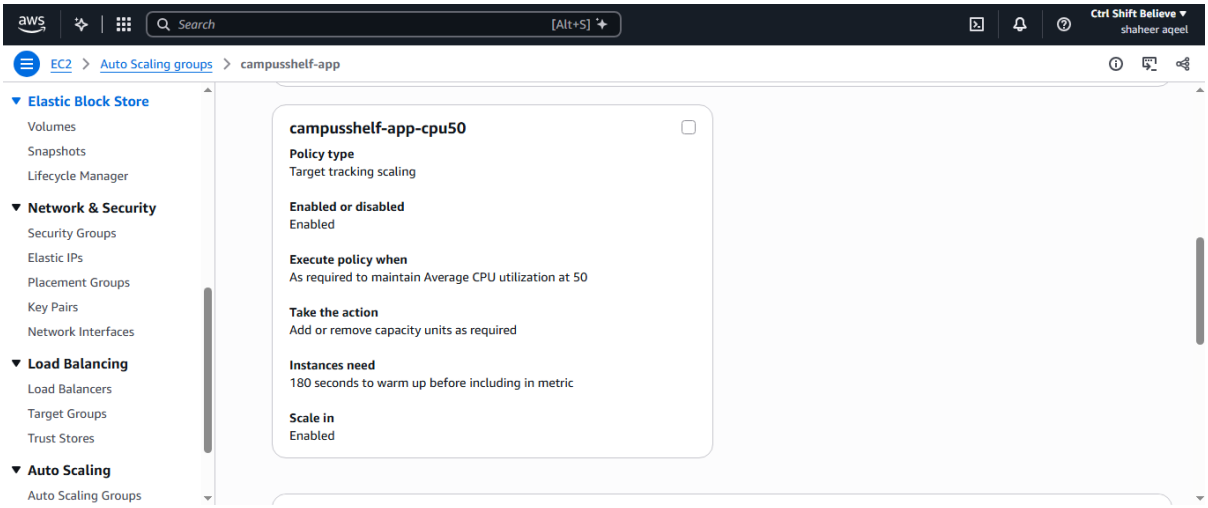


Figure 60: *Scaling policy campusshelf-app-cpu50*

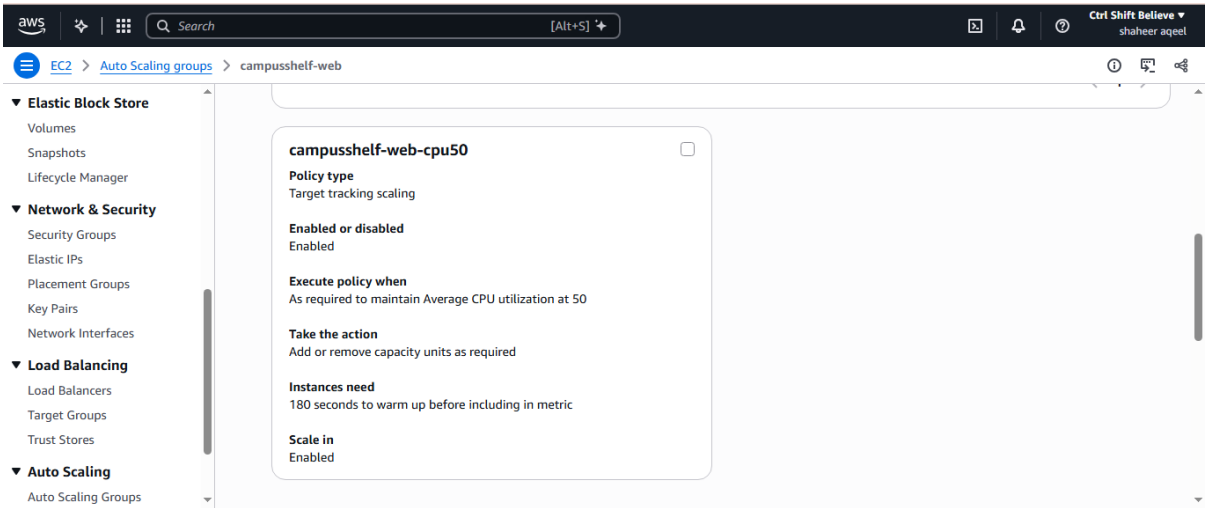


Figure 61: *Scaling policy campusshelf-web-cpu50*

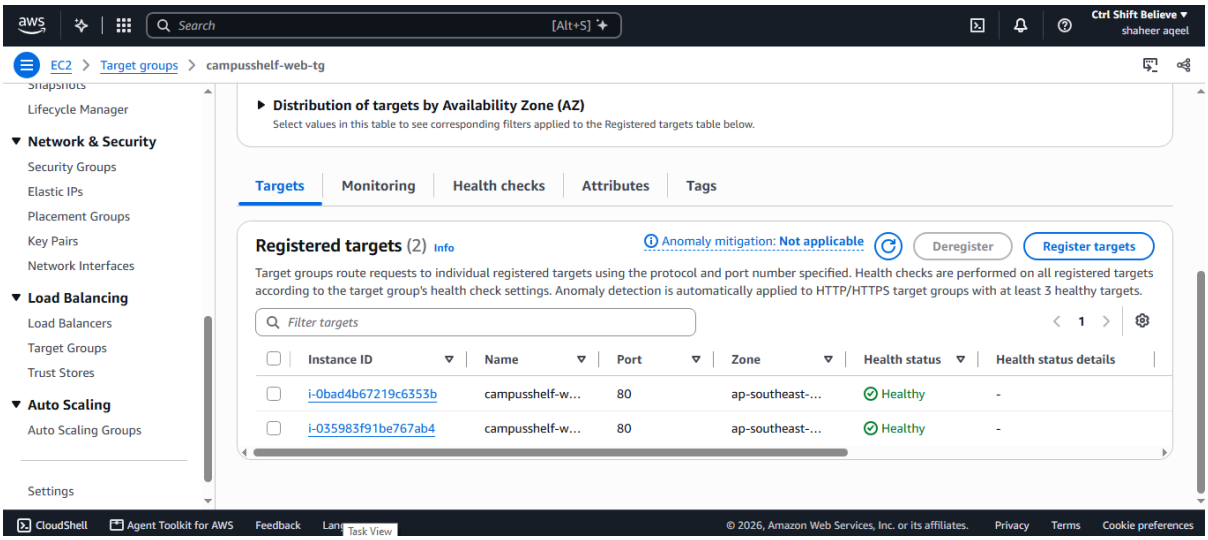


Figure 62: *Before the failure test: two healthy web targets*



Figure 63: *Web logs: requests arriving from both ALB nodes (10.0.1.x and 10.0.2.x)*

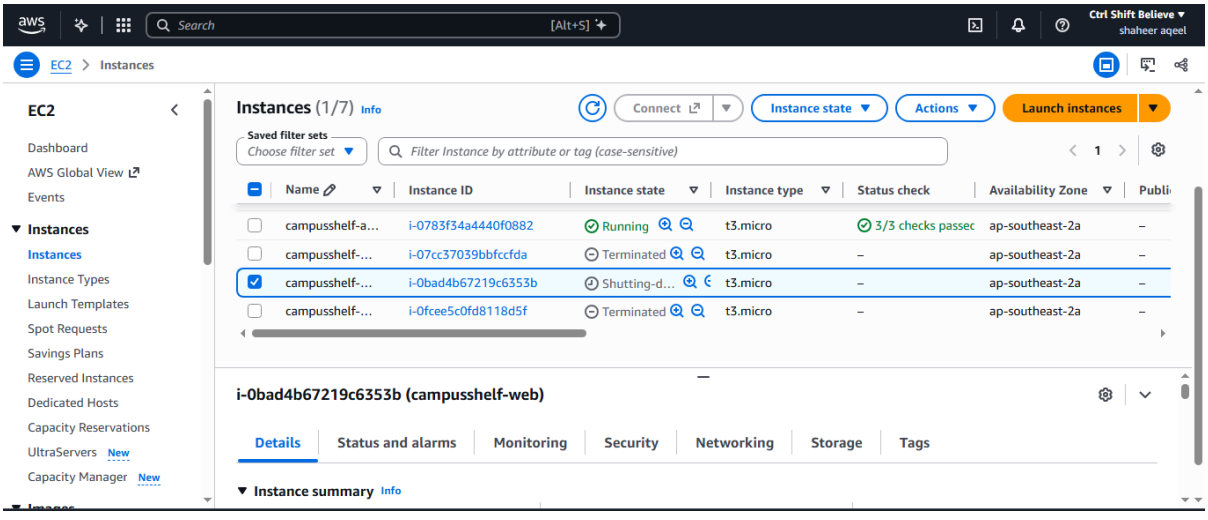


Figure 64: *Failure test: one web instance shut down*



Figure 65: *Continuous curl loop during the failure test*

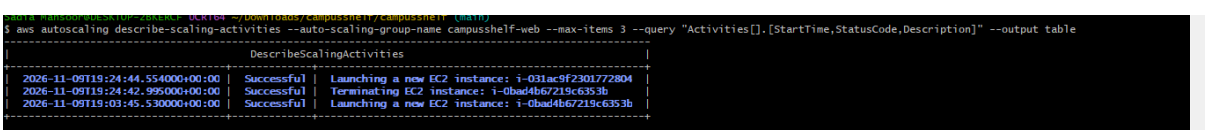


Figure 66: *Self-healing: the ASG terminated the failed instance and launched a replacement*

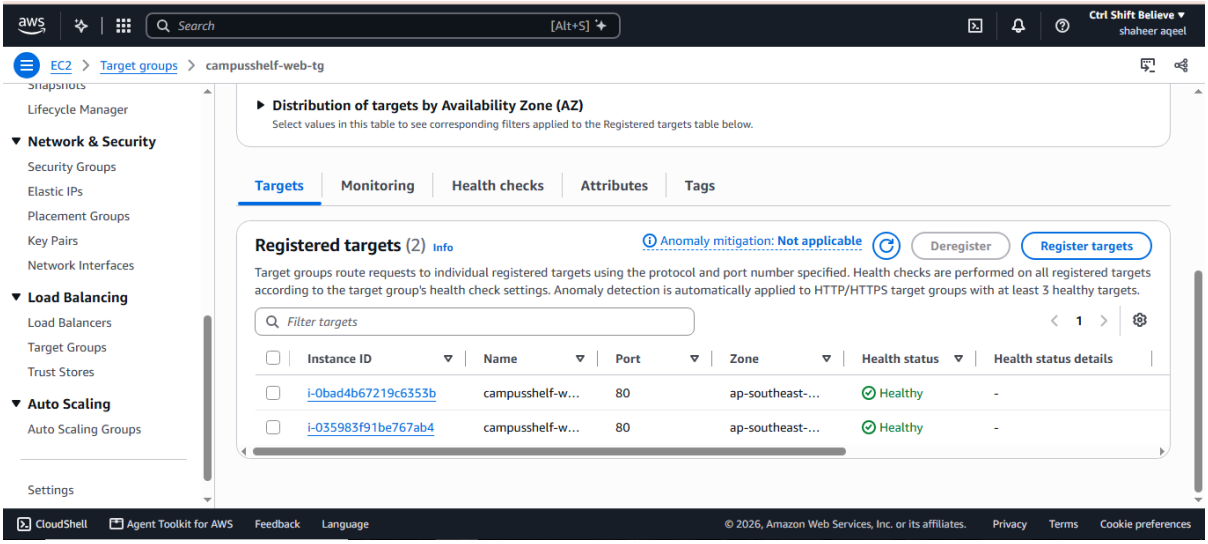


Figure 67: *After recovery: both web targets Healthy again*

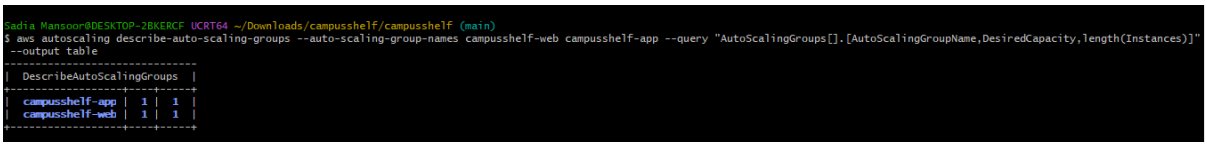


Figure 68: *ASG desired and in-service capacity after recovery*

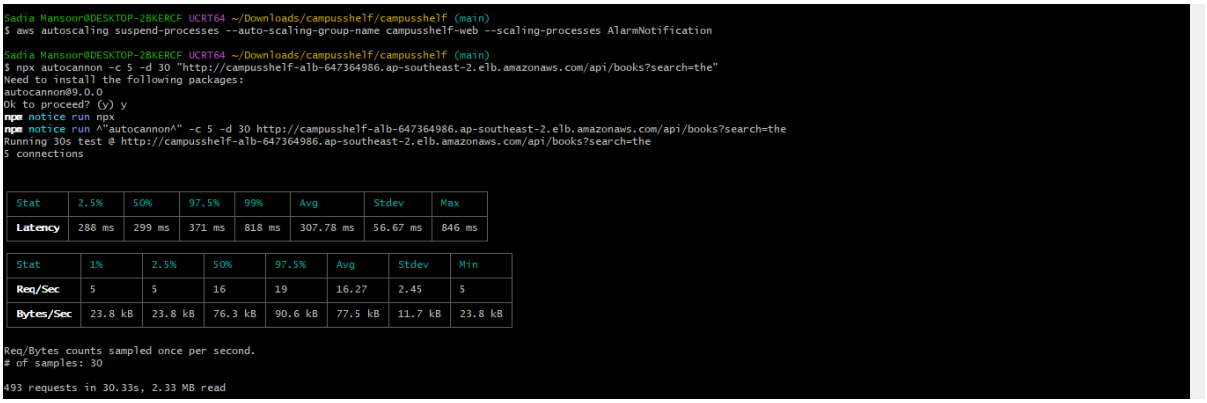


Figure 69: *Load test with autocannon (5 connections, 30 s): 493 requests*

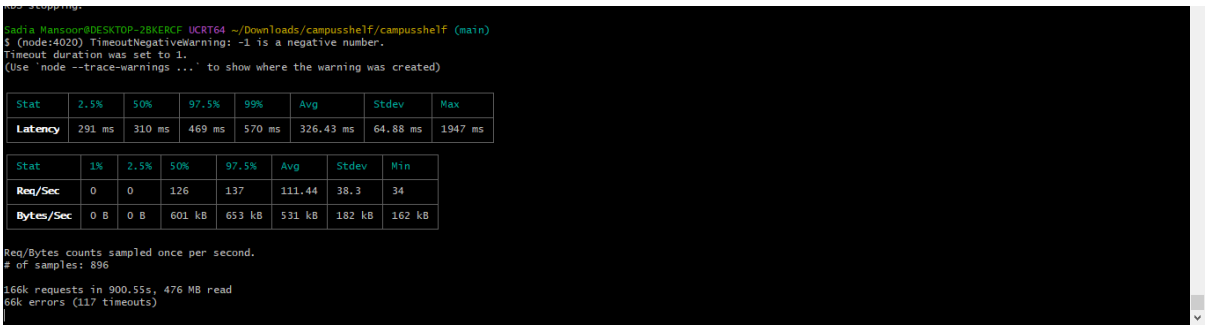


Figure 70: *Extended load test (900 s): 166k requests*

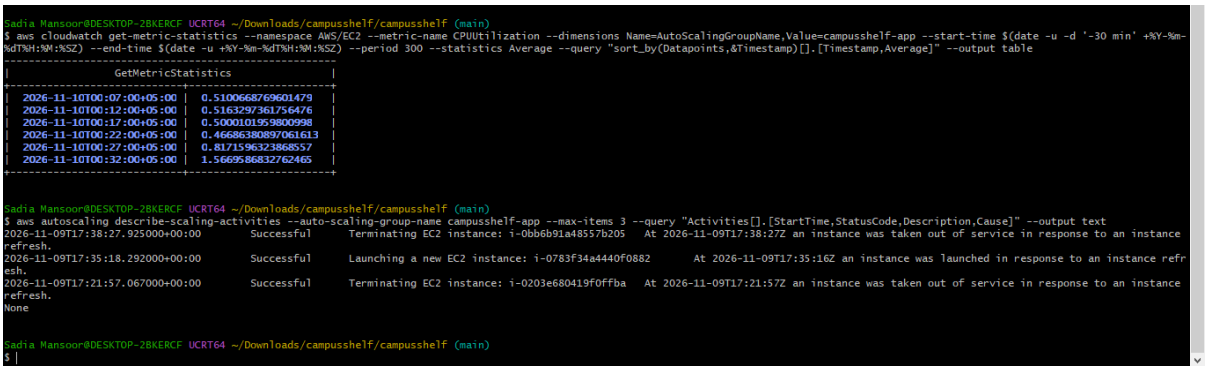


Figure 71: *App ASG CPU utilisation during testing*

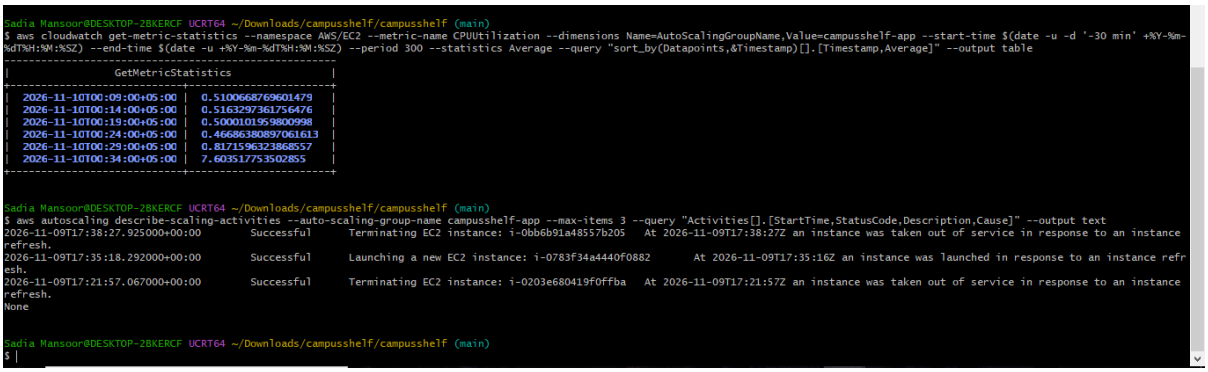


Figure 72: *App ASG CPU metrics and scaling activities*

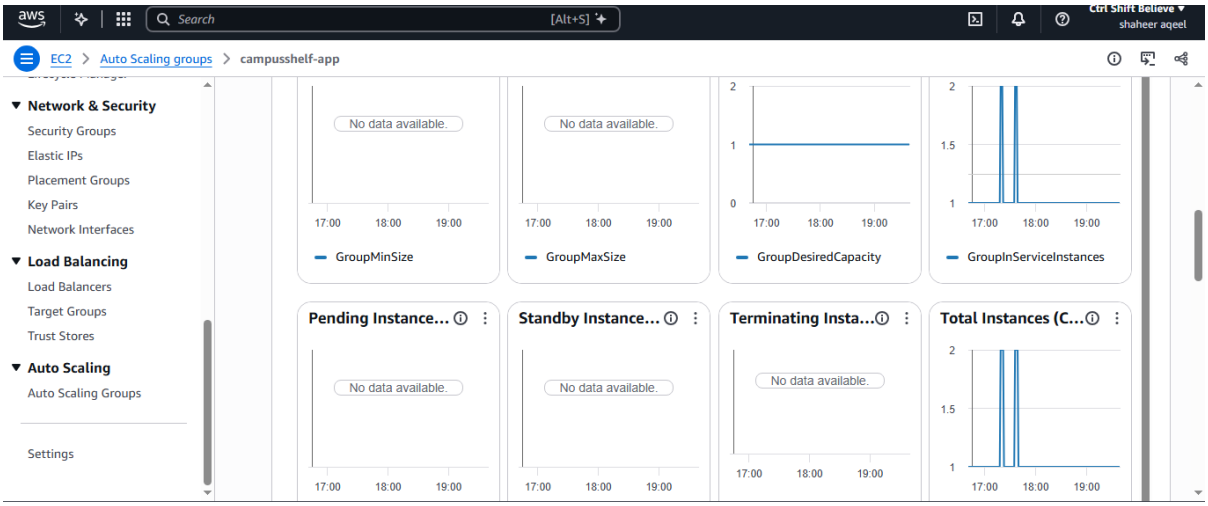


Figure 73: *Auto Scaling Group monitoring graphs*